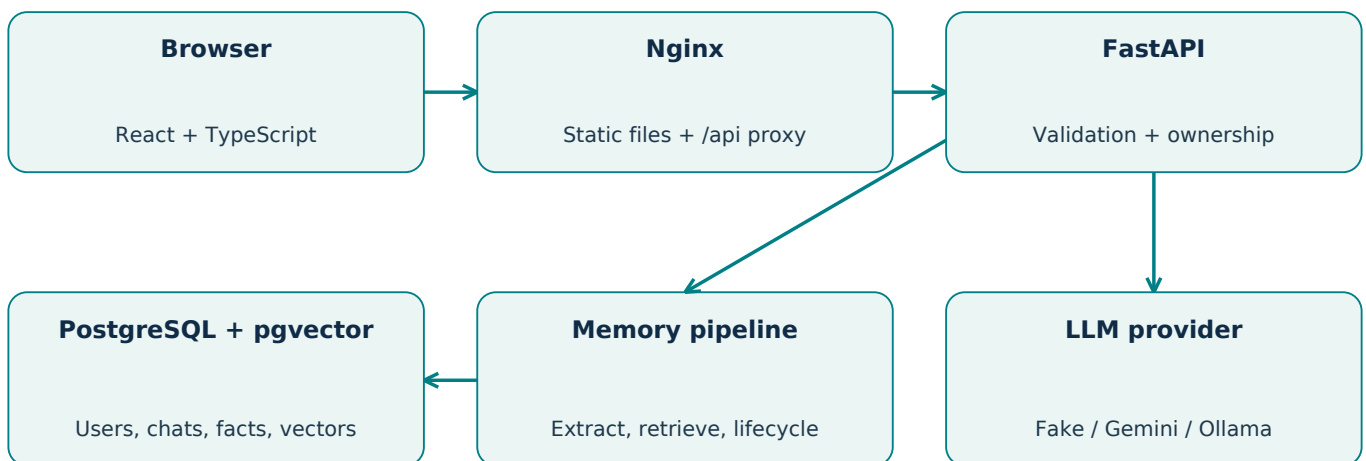


MEMORYAI

YOUR PROJECT. UNDERSTOOD.

The visual interview preparation handbook

From web fundamentals to authentication, streaming AI, user-controlled memory, database design, deployment, debugging and architectural tradeoffs.



Memory selection is application logic; the LLM does not directly read the database.

A study course for readers who are new to the stack, with worked examples, decision comparisons, interview answers and practice exercises.

Repository: [lingudusailokesh/memoryai](https://github.com/lingudusailokesh/memoryai) Edition: 7 October 2026 • inspected branch `fix/cross-chat-preference-recall`, commit `e945012`

The recall fix is included in this code snapshot. Its production rollout is not verified. Main, a review branch and a deployed container can represent different versions.

Contents

01 • How to study this guide	4
02 • What MemoryAI is, and why build it?	5
03 • Product map and scope	6
04 • The web, from zero	7
05 • Programming foundations: Python and TypeScript	8
06 • React, routing, hooks and the frontend toolchain	9
07 • FastAPI and backend responsibilities	10
08 • Architecture and separation of concerns	11
09 • Database model: what is stored where?	12
10 • SQL, transactions, ORM and indexes	13
11 • Migrations and schema evolution	14
12 • Authentication concepts before the code	15
13 • Registration, login and session lifecycle	16
14 • Security controls and their boundaries	17
15 • Password recovery and secret configuration	18
16 • A chat turn, end to end	19
17 • Streaming: SSE, cancellation and regeneration	20
18 • LLM providers and reliable API integration	21
19 • AI memory, RAG and fine-tuning	22
20 • Embeddings and cosine similarity, from zero	23
21 • Topic retrieval and ranking	24
22 • pgvector, HNSW and alternatives	25
23 • The cross-chat preference-recall bug and fix	26
24 • Writing memories: extraction and validation	27
25 • Deduplication and conflicting facts	28
26 • Lifecycle, consent and deletion semantics	29
27 • Prompt construction and prompt-injection risk	30
28 • Privacy, export and multi-user isolation	31

29 • Study planner: LangGraph without hype	32
30 • Analytics, search and what the numbers mean	33
31 • Testing strategy and evidence	34
32 • Evaluation: how to prove memory quality	35
33 • Docker, Nginx and Render deployment	36
34 • Local development: verified paths and gotchas	37
35 • Deployment debugging: 502, readiness and 401	38
36 • How to investigate a missing memory in production	39
37 • Scaling the current design	40
38 • Cost, latency and reliability tradeoffs	41
39 • Technology choices: frontend and backend alternatives	42
40 • Technology choices: storage, memory and operations	43
41 • A candid implementation review	44
42 • Code walkthrough: where to look under pressure	45
43 • Your interview pitch and demonstration	46
44 • Interview answers: product and architecture	47
45 • Interview answers: backend, database and concurrency	48
46 • Interview answers: auth and security	49
47 • Interview answers: memory and AI	50
48 • Interview answers: deployment, tests and scale	51
49 • Scenario drills with reasoning checkpoints	52
50 • Hands-on exercises that make you interview-ready	53
51 • Glossary: web, data and security	54
52 • Glossary: AI and operations	55
53 • Seven-day preparation plan and self-assessment	56
54 • One-page recall sheet and source trail	57

01 • How to study this guide

Your objective

Understand the project well enough to explain a request from the browser to the database and model, defend its decisions, and diagnose failures without guessing. You do not need to memorize every line. You need a correct mental model, concrete evidence, and the ability to reason when the interviewer changes the scenario.

This is a code-grounded study guide, not a claim that every proposed feature is complete or that every production problem has been resolved. It reflects the inspected branch at commit e945012. Older documentation describes stages of development and can disagree with the current implementation; where that happens, this guide follows the code.

Three labels to keep in mind

Label	What it means
Implemented	A behavior or structure exists in the inspected source. It may still have limitations.
Verified	A specific check was run and passed. A local test does not establish every production condition.
Proposed	A design improvement or alternative, not a feature you should claim is already deployed.

Use four passes

- **Pass 1:** read the product and foundations chapters. Explain frontend, backend, database, HTTP and an LLM in ordinary language.
- **Pass 2:** redraw authentication, chat, memory retrieval and memory writing without looking. Each arrow should say what travels across it.
- **Pass 3:** compare alternatives. For every choice, state the constraint, benefit, cost and condition that would make you change it.
- **Pass 4:** practice the answer bank aloud. Then answer the follow-up questions without reading the sample response.

A useful answer pattern

Use **problem** → **decision** → **mechanism** → **evidence** → **limitation**. For example: “A model forgets facts across independent chats. I store small facts by user and retrieve relevant ones before generation. The fact enters the system context as data. A regression captures that context. Retrieval can still miss facts, so I evaluate recall and expose memories used.”

No handbook can guarantee answers to every possible interview question. This one gives you the concepts and source references to reason about unfamiliar follow-ups rather than memorize scripts.

02 • What MemoryAI is, and why build it?

The problem in simple terms

An ordinary chatbot sees only the context supplied in its current request. A person may tell it “I am learning Python” in one conversation and have to repeat that in the next. Sending every previous conversation would be expensive, noisy and potentially intrusive.

MemoryAI extracts short facts from completed conversations, stores them separately, and recalls relevant facts in later chats for the same user. Users can inspect, edit, approve, reject, restore and delete memories. The application also records which memories were supplied for a reply. This makes personalization visible and controllable rather than hidden in an opaque prompt.

What the project demonstrates

It combines a multi-user web product with an AI application. The interesting engineering is not just calling a model API: it is isolating users, handling expiring sessions, streaming without exhausting database connections, validating model-produced facts, resolving conflicts and debugging deployment dependencies.

A concrete example

A user says “I prefer morning study sessions.” After a completed reply, an extraction task may save that as a preference. In another chat, a question about study scheduling can retrieve the fact and give it to the model. The model still generates the answer; the fact is additional evidence, not a fixed scripted response.

Approach	Advantage	Limitation
Ordinary isolated chat	Simple and little retained profile data	User repeats context across chats
Send all history	More raw evidence	Context limits, cost, privacy and irrelevant details
Selective memory	Compact, inspectable personalization	Extraction and retrieval can make mistakes

Why this software rather than a generic notes app?

A notes app stores what you explicitly write and lets you search it. MemoryAI adds automatic fact extraction and contextual use during generation. That is useful for ongoing study goals and preferences, but it needs consent and correction because the system may infer a fact incorrectly.

Interview answer: “I built MemoryAI to explore user-controlled persistent context. The value is remembering relevant facts across chats while letting users see and remove what the system uses.”

03 • Product map and scope

What a user can do

Area	Main behavior
Authentication	Register, login, refresh a session, logout and request/reset a password
Chat	Create conversations, send streamed messages, stop a reply, retry or regenerate the newest turn
Memory	View active facts, search, edit, delete, handle pending suggestions and restore superseded facts
Preferences	Global memory switch, Auto/Ask mode, profile fields and stored response preferences
Transparency	View the saved snapshot of memories supplied for an assistant reply
Dashboard	Counts, recent memories, activity and a deterministic focus suggestion
Analytics	Aggregate messages, memory creation and usage-event fields
Privacy	Export local application data or delete the authenticated account
Study planner	A two-node LangGraph workflow: retrieve facts, then produce a plan

What it is not

It is not a model trained from scratch. The application orchestrates existing LLM providers and an optional local embedding model. It does not fine-tune Gemini on each user. It is not a general autonomous agent that executes arbitrary tools or a production-scale distributed system with durable jobs and shared quotas.

A critical scope distinction

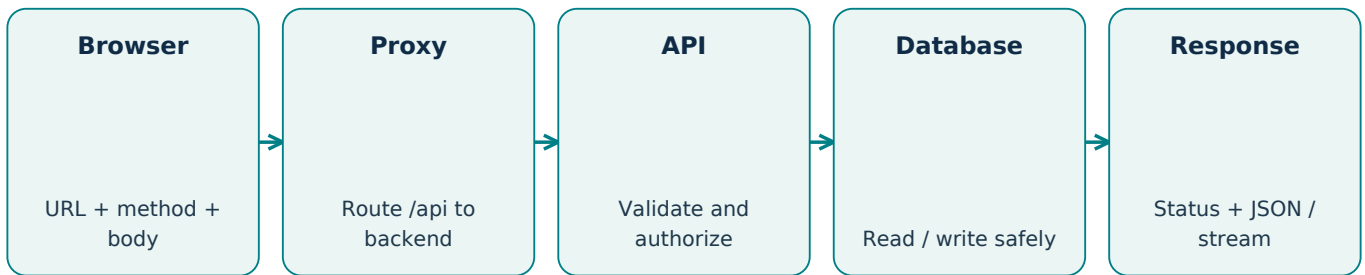
The custom provider has the rich lifecycle implemented in the local memories tables. Fake and Mem0 providers expose the same interface but have different storage and capabilities. A feature present in the interface does not imply identical semantics in every provider. Switching providers does not migrate the user's memories automatically.

Explain the value without exaggeration

Say the project supports persistent, user-scoped memory and a visible context audit. Do not claim perfect recall, complete secret detection, measured cost savings, or universal production reliability. Be ready to show an actual route, test or model-context capture for each assertion.

Self-check: Explain the difference between a conversation, a message, a saved memory, and a snapshot of memories used. They are four different records with different lifetimes.

04 • The web, from zero



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

Browser, server and database

The **browser** runs JavaScript and draws the interface. The **server** accepts requests and enforces rules. The **database** persists information after a process restarts. A browser should not receive database credentials or decide which user's rows it may access.

A **URL** identifies a resource, such as `/api/conversations`. The **HTTP method** describes the action: GET reads, POST creates or triggers an operation, PATCH changes selected fields, and DELETE removes. JSON is a structured text format for exchanging values. HTTPS is HTTP protected by TLS encryption and server identity checks.

Request and response anatomy

```

POST /api/auth/login
Content-Type: application/json
{"email": "student@example.com", "password": "<private>"}

Response: 200 + access token + refresh cookie
Wrong credentials: 401 + generic error detail
  
```

The password shown above is a placeholder, not something to put in logs. A status code describes the result; the response body contains details. A successful network connection can still produce an application rejection.

Status codes you must recognize

Code	Meaning in this project
200 / 201 / 204	Success / created / success without response body
400 / 422	Invalid reset link / invalid structured input
401 / 404 / 409	Unauthenticated or wrong credentials / not owned or absent / conflict
429	Request limit or budget reached
502 / 503	Upstream gateway failure / unavailable dependency or explicit availability error

A 401 with “Incorrect email or password” is fundamentally different from a gateway 502. Diagnose the layer that produced the response before changing code.

05 • Programming foundations: Python and TypeScript

Why two languages?

Python implements the server and AI orchestration. TypeScript implements the browser client. Browsers execute JavaScript; TypeScript is checked and compiled into JavaScript before deployment. Python and TypeScript communicate through HTTP/JSON, not by importing each other's files.

Concepts that recur throughout the code

Concept	Meaning and MemoryAI example
Function	A named operation; <code>hash_password</code> converts a password into a verifier
Class	A type with data/behavior; <code>ChatService</code> coordinates a turn
Interface / abstract base class	A contract; every LLM provider supports streamed chat
Type annotation	Documents and checks expected values; a UUID is different from arbitrary text
Exception	Signals failure; invalid credentials become a controlled 401
Async / await	Lets a process pause on I/O and handle other work while waiting
Generator	Produces values progressively; the chat service yields token events
Dependency injection	Supplies a collaborator; tests substitute a fake model without changing routes

Async is not a magic speed switch

An asynchronous database or HTTP call releases control while it waits. A CPU-heavy function executed directly in an async route can still block the event loop. `FastEmbed` runs through `asyncio.to_thread`; password hashing is currently synchronous. For higher concurrency, benchmark hashing and consider bounded thread offloading rather than assuming all work is nonblocking.

Type safety has limits

TypeScript catches many mistakes before shipping, but cannot prove a live server returned the expected JSON. `Pydantic` validates server inputs at runtime. Database constraints are another line of defense. Each protects a different boundary.

Interview answer: “I use Python for typed async API and AI orchestration, and TypeScript for safer browser code. Their contract is HTTP payloads; compile-time types do not replace runtime validation.”

06 • React, routing, hooks and the frontend toolchain

React in plain English

React describes the screen as a function of state. When state changes, React updates the relevant UI. A **component** is a reusable view. A **hook** packages stateful behavior. MemoryAI separates page components from hooks that handle authentication and chat.

Know the actual files

- `hooks/useAuth.tsx` holds the user and loading/authenticated/anonymous status. Login stores an access token in memory, then fetches `/api/me`.
- `hooks/useChat.ts` handles conversation loading, optimistic messages, streamed updates, stopping and regeneration.
- `services/api.ts` adds authentication headers, refreshes sessions, checks readiness and parses streamed events.
- `ProtectedRoute.tsx` controls client navigation. It is a usability feature; server authorization remains mandatory.

Stack choices

React Router maps paths such as `/chat/:id` to pages without a full reload. **Vite** provides a fast development server and builds static assets. **Tailwind CSS v3** provides utility classes; CSS variables provide theme tokens. **Lucide** supplies icons, **Recharts** draws analytics and **react-markdown** renders assistant text.

Why not Next.js here?

The core experience is a signed-in interactive application, and the backend is already Python. A Vite SPA keeps the browser build simple and avoids adding a second server-rendering runtime. Next.js would become attractive for public SEO pages, server rendering or an integrated JavaScript backend. It is not inherently worse; it solves additional problems not central to this design.

Why not Redux immediately?

Authentication context and a few focused hooks are enough for the current state model. Redux or a query-cache library could help with complex shared state and cache invalidation, but introduces concepts and dependencies. Re-evaluate when duplicated fetching and cross-page consistency become real problems.

Drill: Describe what happens to the access token when the page refreshes. It disappears from memory; the refresh cookie allows the application to ask the server for a new token.

07 • FastAPI and backend responsibilities

What FastAPI supplies

FastAPI maps HTTP routes to Python functions, integrates Pydantic validation, resolves dependencies and generates OpenAPI documentation. Uvicorn is the ASGI server that actually listens for network requests and invokes the application. FastAPI is the framework; Uvicorn is the serving process.

The application entry point

`app/main.py` assembles the routers, middleware and startup lifespan. Startup constructs the configured LLM, memory provider and email sender so obvious misconfiguration fails early. Middleware adds security headers and a request ID. `/api/health` describes configured providers; `/api/ready` performs a database SELECT 1.

Boundary validation

`RegisterIn` requires a valid email and password length between 8 and 128 characters. `MessageIn` caps a message at the configured limit, default 8,000 characters. Pydantic rejects structurally invalid input before the service operates on it. It does not verify an account exists or that a database is reachable.

Why FastAPI rather than Flask or Django?

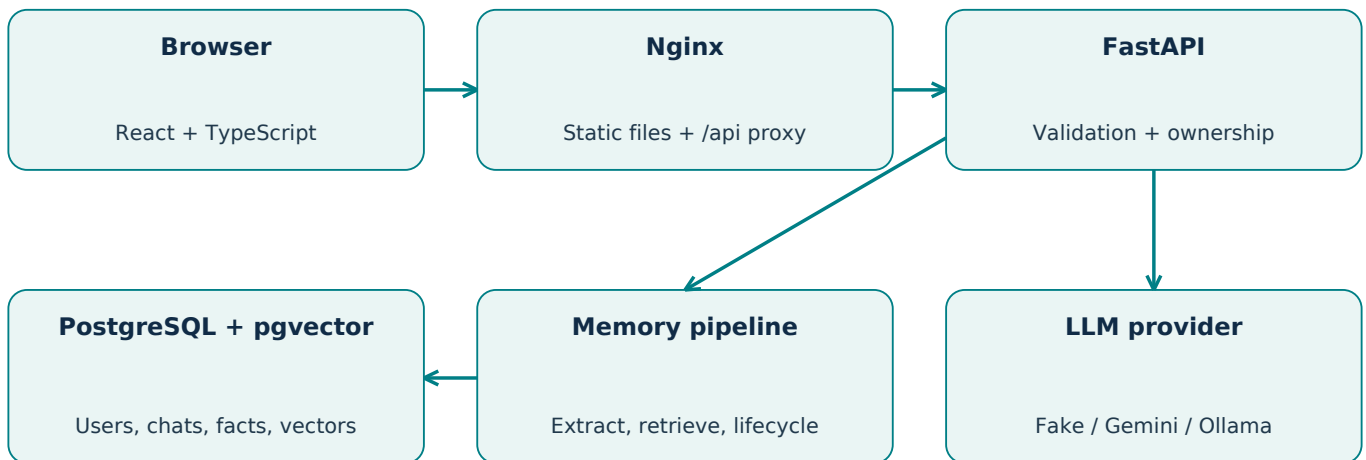
FastAPI fits typed async request handling and streaming model output. Flask is lightweight and excellent for many APIs, but its traditional ecosystem is synchronous. Django offers a rich ORM, admin and built-in account tooling, but this project uses a smaller API-centric structure and SQLAlchemy. If administrative workflows and conventional CRUD dominated, Django would be a strong alternative.

Important dependency pattern

A route declares `Depends(get_current_user)`, `Depends(get_session)` or `Depends(get_memory_provider)`. Production uses configured objects. Tests override dependencies with isolated database sessions and controlled providers. This keeps business logic testable, but also means tests can bypass production dependencies unless dedicated integration checks exist.

Do not say “FastAPI secures every route automatically.” A route is protected only if it uses the authentication dependency and its data queries enforce ownership.

08 • Architecture and separation of concerns



Memory selection is application logic; the LLM does not directly read the database.

The layers

API routes interpret HTTP and map known errors to status codes. **Services** coordinate business operations, such as issuing tokens or constructing a chat turn. **Repositories** perform user-scoped database access. **Providers** adapt external or interchangeable systems such as Gemini, Ollama, Mem0 and embeddings.

For example, the login route does not implement Argon2. It invokes `AuthService.login`, which looks up a normalized email through `UserRepository` and calls security helpers. The chat service knows the LLM contract, not Gemini's URL or response format.

Why a modular monolith?

The backend has distinct modules but runs as one application. This reduces network calls, deployment complexity and distributed transactions. It is easier to understand and test while requirements are evolving. Microservices would add independent scaling and ownership, but also retries, authentication between services, versioned contracts and operational burden.

Design principles, without buzzwords

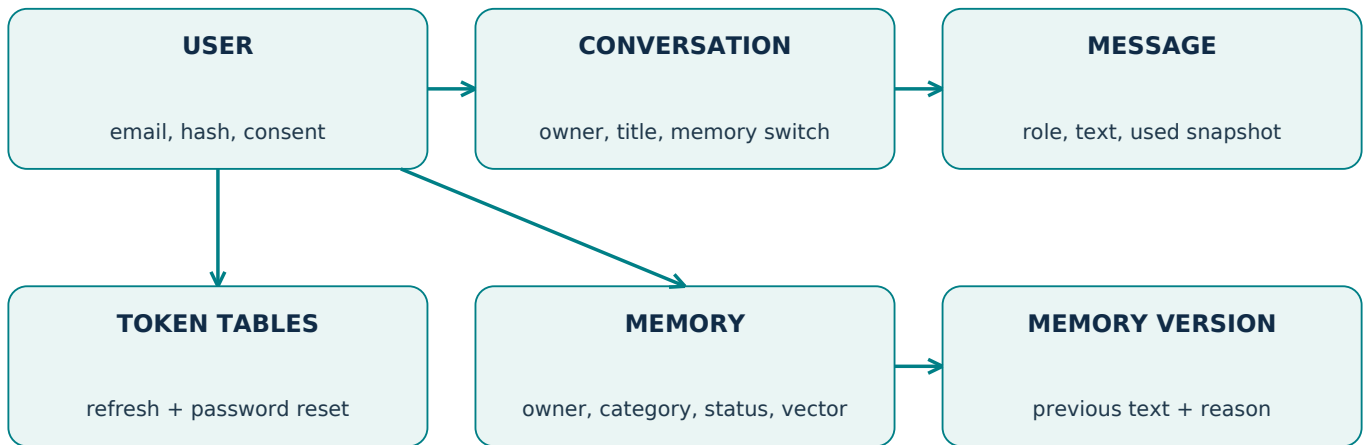
A module should have a coherent responsibility. Depend on a small contract rather than a vendor implementation. Keep long-running external calls outside database transactions. Place authorization near data access, not just in a page. These choices reduce the number of places that must change when a provider or feature changes.

Tradeoff

Repositories and provider interfaces create more files than a single script. They are worthwhile here because multiple users, providers and memory lifecycle operations need consistent rules. For a tiny disposable prototype, this structure might be unnecessary.

Interview exercise: Trace a single fact from the extraction provider through `MemoryRepository` and back through `ChatService._recall`. Explain where user ownership is applied and where the final prompt is assembled.

09 • Database model: what is stored where?



Arrows: parent to child. Usage events belong to users; retrieval events belong to messages.

Read this relationship map

A user owns many conversations and memories. A conversation owns messages. A memory owns historical versions. Refresh and password-reset tokens belong to the user. Usage events belong to users; memory-retrieval events belong to assistant messages.

Important design details

Record	Why it exists
User	Identity, password verifier and persistent preference/consent fields
Conversation	Title and per-chat memory switch; establishes ownership for messages
Message	Role, text, creation time and a JSON snapshot of supplied memories
Memory	A compact fact, category, importance, lifecycle status and embedding
MemoryVersion	Previous text and reason for a change; supports inspection/history
RefreshToken	Hash, expiry and revocation state; permits session rotation/logout
PasswordResetToken	Hash, expiry and single-use state for recovery
UsageEvent / MemoryRetrieval	Analytics and retrieval event records, not billing proof

UUIDs and authorization

UUIDs are large random-looking identifiers suitable for URLs and distributed ID generation. They reduce easy enumeration, but they do not authorize anything. Every relevant lookup must still constrain the authenticated user. “We use UUIDs, so users cannot access each other” is an incorrect interview answer.

10 • SQL, transactions, ORM and indexes

Minimum SQL vocabulary

A table contains rows and columns. A primary key identifies a row. Foreign keys connect records; constraints prevent invalid states. Indexes accelerate selected lookups but cost storage and write work. Transactions group operations consistently.

```
SELECT content FROM memories
WHERE user_id = :authenticated_user
      AND status = 'active'
ORDER BY created_at DESC
LIMIT 5;
```

The parameter is bound by the database library, not pasted into SQL. Ownership belongs in the query, not a browser-side filter.

SQLAlchemy in this project

The ORM maps Python classes to tables. An AsyncSession tracks objects and changes; commit makes writes durable. `expire_on_commit=False` avoids automatically expiring loaded attributes. Repositories still refresh rows when server-generated fields could otherwise be stale or unavailable during serialization.

Atomicity has boundaries

The custom memory pipeline inserts, merges and supersedes facts in one transaction per exchange. But an assistant message is committed before usage/retrieval events, and memory deletion and snapshot scrubbing are separate operations. “Everything is transactional” overstates the implementation.

Indexes and query plans

User-ID indexes support isolation-scoped queries. The message (`conversation_id`, `created_at`) index supports ordered history; HNSW supports approximate vector search. Defining an index does not prove PostgreSQL uses it. Inspect `EXPLAIN ANALYZE` on representative non-sensitive data before claiming query speed.

Deletion relationships

Messages are children: deleting a conversation cascades to messages. A saved memory's source conversation is attribution: deletion sets its source pointer to null, preserving the independent fact. A memory-retrieval event holds a text identifier rather than a foreign key to the mutable memory. Its supplied text is separately copied into the message snapshot until deletion explicitly scrubs it.

Why PostgreSQL rather than MongoDB?

Ownership, unique emails, relationships and multi-row changes fit relational storage. `pgvector` also keeps vectors in the same system. MongoDB can suit document-heavy evolving schemas, but would not remove authorization or consistency requirements.

A parameterized query prevents SQL injection, not unauthorized access. A valid query still leaks data if it omits the owner's predicate.

11 • Migrations and schema evolution

Why not just create tables on startup?

As the application changes, existing databases already contain valuable rows. A migration records how to evolve their structure without pretending they are empty. Alembic tracks revision history and executes upgrade operations. The backend container runs `alembic upgrade head` before starting Uvicorn.

What the migration chain introduces

The repository includes revisions 0001 through 0008: initial identity/chat tables, refresh tokens, used-memory snapshots and vector support, custom memories, reset tokens, memory lifecycle, product settings/telemetry, and a repair migration for missing product schema.

The 0008 lesson

A database can be stamped with revision 0007 while some expected DDL is absent. A version label is not proof of physical schema completeness. Revision 0008 inspects columns/tables/indexes and adds missing objects. It does not rewrite old migration history or delete production rows. Its downgrade is intentionally a no-op because it cannot know which objects existed before repair.

Safe rollout reasoning

For a schema-changing feature, design compatibility between old and new code. Prefer an expand/migrate/contract sequence: add a compatible column, deploy code that can use it, backfill safely, and only later remove obsolete structures. Back up and practice restoration. With multiple replicas, use one migration runner or locking; having every process race to migrate is unsafe.

Interview answer

“I use Alembic because the database outlives a code deploy. A deployment must be compatible with the current schema, and a revision stamp must be checked against reality when evidence suggests drift. I prefer forward corrective migrations over silently editing a migration that may already have run.”

Do not run table-drop integration tests against production. The real pgvector suite deliberately creates and drops tables and requires a dedicated disposable test database.

12 • Authentication concepts before the code

Identity versus permission

Authentication answers “Who is making this request?” **Authorization** answers “May that user access this object?” A valid token for user A must not allow access to user B's conversation. The server derives the user from a verified token, not a user ID submitted by the client.

Password hashing is not encryption

Encryption is reversible with a key. Password hashing produces a verifier designed to be expensive to guess. Argon2id adds a salt and memory cost, so identical passwords need not have identical stored hashes and bulk guessing is costly. The server checks the supplied password against the stored verifier; it cannot recover the original password.

JWT in plain language

An access JWT contains claims such as user ID and expiry, with a signature. A signature proves integrity, not secrecy: a typical JWT payload is readable. MemoryAI uses HS256 with an explicitly configured secret. Verification must constrain the accepted algorithm and validate required claims, expiry and token type.

Cookies and bearer headers

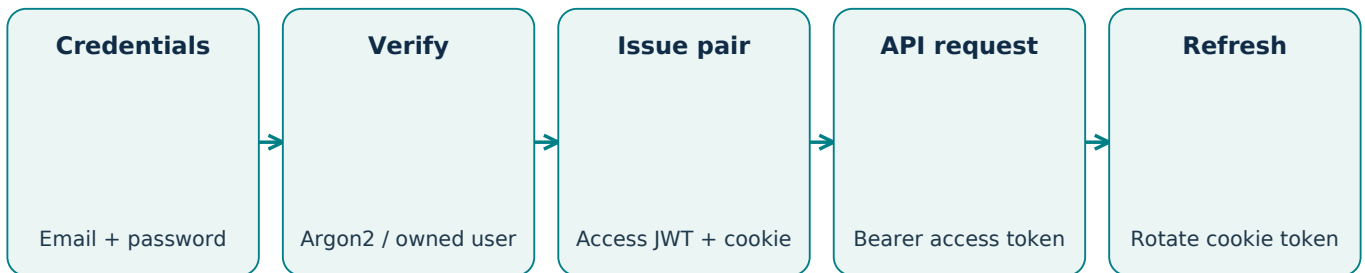
A bearer access token is sent in `Authorization: Bearer ....` A cookie is stored by the browser and sent automatically to matching requests. The long-lived refresh cookie is `httpOnly`, so JavaScript cannot read its value; it is `SameSite=Lax` and scoped to `/api/auth`. Production HTTPS requires the `Secure` flag.

Why two tokens?

The short access token limits exposure and authenticates normal requests. The opaque refresh token permits continued sessions without re-entering a password, while its stored hash and revocation state allow rotation. Both are secrets when issued, even though their formats differ.

Avoid this false claim: “JWT means there is no database query on authenticated requests.”
`get_current_user` still loads the user, so deleted accounts and current user existence are checked in the database.

13 • Registration, login and session lifecycle



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

Register and login, step by step

Registration validates the email/password, normalizes the email in the repository, hashes the password and creates the user. A successful registration returns 201 and issues an access token plus refresh cookie. Login finds the normalized email and verifies the password; unknown email and wrong password both produce the same 401 detail.

Email normalization trims and lowercases the entire address. This is practical for this application, but the theoretical email local-part case rules are more nuanced. Passwords are not lowercased or trimmed: case, whitespace and Unicode are part of the secret.

Refresh rotation

A raw refresh token is random and only its SHA-256 hash is stored. Refresh consumes an unrevoked, unexpired token and issues a new pair. The repository uses an atomic update/returning pattern so simultaneous uses cannot both consume the old token. The frontend shares one in-flight refresh promise to avoid its own requests racing each other.

Logout and expiry

Logout consumes the current refresh token, deletes the cookie and clears client state. It does **not** instantly revoke a previously issued access JWT. That JWT may remain usable until expiry, default 15 minutes, unless the account itself is no longer present. A centralized token-version check or denylist would change this tradeoff.

Page reload

The in-memory access token disappears. AuthProvider attempts cookie refresh, then fetches `/api/me`; it becomes authenticated or anonymous. The availability guard now waits for database readiness before login, register and refresh calls. It never automatically replays an authentication POST after a gateway failure.

A local account is not automatically a production account. Account existence depends on the database that processed registration, not on whether the same repository code is deployed.

14 • Security controls and their boundaries

What is implemented

Owner-scoped repositories; Argon2id password hashing; constrained JWT verification; hashed refresh/reset tokens; httpOnly refresh cookies; a required JWT secret; input length checks; security headers; best-effort sensitive-memory filtering; and generic credential errors. Other users' objects are generally represented as 404 to avoid confirming existence.

What these controls do not prove

An httpOnly cookie reduces direct theft of that cookie through JavaScript, but XSS can still act as the user or read the in-memory access token. SameSite=Lax reduces cross-site cookie POST exposure but is not a substitute for reviewing all cross-origin paths. HTTPS encrypts transport, not every database row at rest. UUIDs do not replace ownership checks.

Risk	Current defense	Remaining work
Guessing passwords	Expensive hashing, generic login error	Auth-specific throttling, monitoring and recovery abuse controls
User data leakage	Ownership in repository queries	Audit every route/provider, add database row-level security if justified
SQL injection	Bound ORM expressions	Avoid unsafe dynamic SQL; validate special query semantics
Stored prompt injection	Facts labeled as data, flattened and capped	Model can still follow malicious data; evaluate adversarial prompts
Request abuse	Chat per-minute limiter and daily budget	Limits are not uniformly applied across auth, regenerate and planner

Registration enumeration

A taken email returns 409. This helps users but reveals registration status. The generic login response does not eliminate this separate enumeration channel. Product UX and abuse resistance must be weighed explicitly.

Safe interview phrasing

“I designed multiple defenses around identity, isolation and secret handling. I would not call the application fully hardened: auth throttling, shared enforcement, consent races and privacy retention still need work.” This is stronger than claiming perfect security without evidence.

15 • Password recovery and secret configuration

Recovery flow

Forgot password returns the same 202 message for known and unknown emails. For a valid account outside cooldown, the service creates a short-lived, hashed token and schedules an email. A reset consumes the token once, hashes the new password, revokes refresh sessions and clears the cookie.

The default reset lifetime is 30 minutes and the account cooldown is 60 seconds. A new reset request invalidates older reset links. The frontend removes the token from the visible browser URL after reading it, reducing history/referrer exposure. Email is sent after the HTTP response; that does not guarantee indistinguishable timing for every server-side lookup.

Email delivery is a dependency

The default EMAIL_BACKEND=console writes email content to development logs rather than delivering mail. Production needs the SMTP sender configured with STARTTLS, credentials and an appropriate sender address. FRONTEND_URL must point to the deployed frontend so reset links return to the correct site. Treat reset-link logs as secrets.

Configuration rules

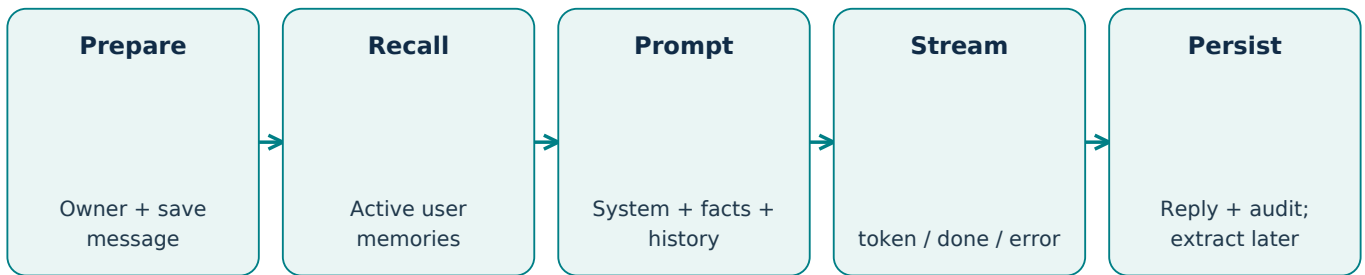
Variable group	Purpose
JWT_SECRET / COOKIE_SECURE	Signing key and production HTTPS cookie policy
DATABASE_URL	The actual database and async-capable driver
LLM_PROVIDER / model credentials	Which generation provider is constructed
MEMORY_PROVIDER / EMBEDDING_PROVIDER	Storage/retrieval implementation and vector representation
EMAIL_BACKEND / SMTP_* / FRONTEND_URL	Real recovery email and correct link destination

Important revocation nuance

Revoking refresh tokens stops session continuation. Already issued access JWTs can remain valid until expiry. The reset path does not maintain an immediate access-token denylist. Explain the bounded exposure honestly rather than saying all access ends instantly.

Never commit .env files, print database URLs or copy real access/reset/refresh tokens into interview slides. Share variable names, redacted examples and boolean diagnostics instead.

16 • A chat turn, end to end



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

Before generation

The route validates the message, authenticates the user and applies the implemented new-message limits. `ChatService.start_turn` checks the conversation owner, saves the user message and loads bounded recent history. It reads the global and conversation memory switches from the database, rather than trusting a client flag.

During recall

After that short preparation session closes, `_recall` calls the configured memory provider with the authenticated user ID and latest query. It allows an 8-second memory timeout. On failure it logs and continues without memory. Retrieved facts are placed into a system prompt as labeled data with flattened whitespace and a per-fact text cap.

During streaming

The LLM receives system context plus history. The service yields tokens incrementally without holding its own preparation session open. The browser updates an optimistic assistant draft. This improves perceived responsiveness: time to first useful text matters even if total generation time is unchanged.

After generation

A complete nonempty reply is saved. Usage events and memory-retrieval records are written, the conversation is touched and a title may be generated. If memory was enabled and the turn is a new exchange, extraction is scheduled in a background task. A done event includes the persisted message ID, optional title and memory count.

Failure boundaries

A failed memory search is not a failed chat. A failed generation produces an error event. A database failure after text was streamed may mean the user saw text that was not fully persisted. A title failure can fall back locally. These operations should be monitored separately rather than collapsed into “AI is broken.”

Implementation caveat: the new-message route opens a session for its daily-budget query, so the intended no-connection-during-stream rule should be audited across route dependencies as well as the chat service itself.

17 • Streaming: SSE, cancellation and regeneration

What SSE means here

Server-Sent Events encode named events as text blocks separated by a blank line. This project uses a POST fetch and a readable response stream, rather than native EventSource, because it sends a JSON message body and bearer authorization header.

```
event: token
data: {"text": "You prefer "}

event: done
data: {"message_id": "...", "memories_used": 1}
```

The frontend buffers chunks until complete event blocks arrive; network chunks do not necessarily align with JSON or token boundaries. It handles token, done and error events. A stream may have HTTP 200 and later contain an application error event, so HTTP status alone is insufficient.

Why not WebSockets?

Each chat request mainly needs one-way server-to-browser output. HTTP streaming is easier to fit behind existing proxies and uses familiar request authentication. WebSockets are appropriate for ongoing bidirectional collaboration, live audio or many interactive events on one connection, but add connection lifecycle and scaling complexity.

Stop and partial persistence

Stop aborts fetch. If text already exists, the server attempts to save the partial reply using `asyncio.shield` so cancellation does not cancel the save itself. A stopped or failed partial reply is not extracted into memory. The UI may not know the final saved message ID, so regeneration first reloads the server's latest message.

Regenerate semantics

Only the newest message may be targeted. An assistant target is deleted and replaced; a user target means retrying a failed answer. Regenerating an assistant is not a new exchange for memory writing. There is no full branch/version history of alternative assistant answers.

Current limitation: newest-message validation is not a per-conversation concurrency lock. Two simultaneous requests can still race. Propose serialization or optimistic versioning when asked about multi-tab correctness.

18 • LLM providers and reliable API integration

A model is an external dependency

The model transforms supplied messages into output. It does not automatically know the project's database, consent settings or prior chats. LLMProvider defines streamed chat and a derived complete operation. Fake, Gemini and Ollama adapters implement this contract.

Provider	Suitable use	Tradeoff
Fake	Offline development and deterministic tests	Echoes/scripted behavior; no realistic reasoning
Gemini	Hosted generation with small model configuration	Network, quotas, rate limits, privacy review and provider dependency
Ollama	Locally served model	Hardware/memory/latency and model operations become your responsibility

Why raw httpx rather than a vendor SDK?

The application already uses httpx, needs explicit streaming control and benefits from easily mocked transports. Adapters translate internal roles into each vendor's payload. A vendor SDK could handle API changes and metadata, but adds a dependency and may obscure retry or streaming behavior. Either can be correct if the boundary remains testable.

Retry policy

Gemini retries 429/503 with exponential backoff and jitter before emitting any text, up to the configured adapter retry count. After output starts, replaying the request could duplicate text or generate a different answer. Transport errors currently become controlled unavailability errors rather than being broadly retried.

Context and model limits

Default settings bound history to 20 messages and output to 800 tokens. Message count is not an exact token budget: a few large messages can still exceed a provider context limit. A production improvement is model-specific token accounting with a prioritization/truncation policy.

Interview answer: "I isolate model-specific payloads and failures in providers. This allows controlled tests and changing the generator without rewriting auth or memory storage. Vendor flexibility still requires evaluating output quality and data-handling policy."

19 • AI memory, RAG and fine-tuning

Four concepts that are often confused

Concept	What it means here
Chat history	Recent messages from one conversation included in the request
Persistent memory	Separately stored, user-scoped facts that can be used across chats
RAG	Retrieve evidence first, then include it in a model generation request
Fine-tuning	Change model parameters through training; MemoryAI does not do this

Why memory is a RAG problem

The application stores knowledge externally, searches it for the current request and gives selected evidence to the model. This is a form of retrieval-augmented generation. The model weights stay unchanged. Correcting or deleting a fact changes later retrieval without retraining a model.

Why not fine-tune on every user's conversations?

Preferences change and users need immediate deletion/control. Per-user fine-tuning would require training data, compute, evaluation, versioning and difficult knowledge removal. It may learn style or domain behavior but is a poor default tool for a few mutable facts. RAG provides an explicit store and context audit; it still has retrieval errors.

Why not a single global profile prompt?

A short static profile can be a useful baseline. Always sending it is simple but exposes facts on irrelevant turns and grows with usage. MemoryAI chooses selective recall and bounded context. Broad profile questions need a different relevance signal than a narrow topic query, which motivated the preference-recall fix.

A healthy model of correctness

You need to test three stages separately: **Did the right fact get stored? Did it get retrieved? Did the model use it appropriately?** An assistant saying “I do not know” is not proof that storage failed. A used-memory chip proves supplied context, not that the model's answer is correct.

Interview answer: “The persistent memory lives in our database, not inside the model. RAG makes that context editable, user-scoped and observable. It avoids per-user retraining, but retrieval and generation still need separate evaluation.”

20 • Embeddings and cosine similarity, from zero

A vector is a list of numbers

An embedding model maps text to a numerical vector so semantically related text tends to have nearby directions. A 384-dimensional vector has 384 numbers; the dimensions are learned features, not one obvious word per position. Similarity is a model-dependent proxy for relevance, not truth or consent.

A tiny illustrative calculation

Take two toy vectors $A = [1, 0]$ and $B = [1, 1]$. Their dot product is 1; their lengths are 1 and $\sqrt{2}$. Cosine similarity is $1 / \sqrt{2}$, about 0.707. A vector at right angles has cosine zero. The real application uses 384 dimensions, but the geometry is the same.

```
cosine(A,B) = dot(A,B) / (length(A) * length(B))
cosine distance = 1 - cosine similarity
```

Fake versus semantic embeddings

FakeEmbedder hashes a set of tokens into vector buckets and normalizes the result. It is deterministic and offline, but it does not know that “like” and “likes” or “coding” and “programming” are related. Hash collisions can also create accidental similarity.

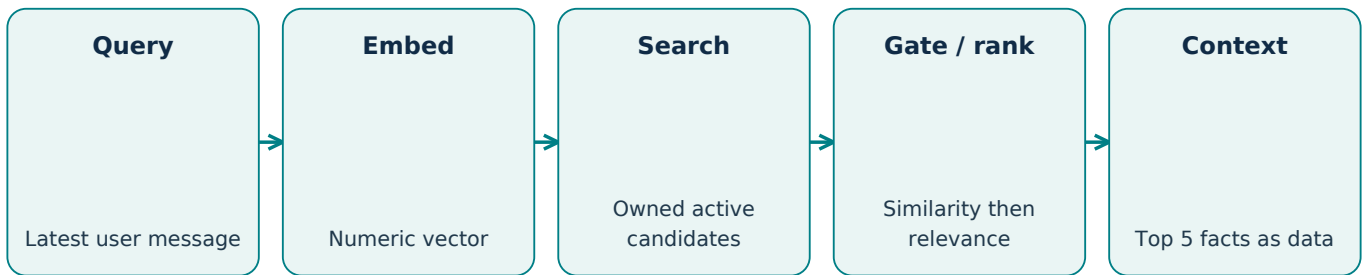
FastEmbedEmbedder optionally runs BAAI/bge-small-en-v1.5 locally through ONNX, with 384 dimensions. It needs the optional embeddings requirements installed and model resources available. The default backend image installs base requirements only; selecting FastEmbed without installing its package is not a valid deployment plan.

Model changes require a migration plan

Two models can both output 384 numbers but represent incompatible spaces. Existing rows store `embedding_model`, yet the normal search does not enforce a matching model before comparing. Re-embed existing rows consistently, record model/version and prevent mixed-space comparisons; changing an environment variable alone is not enough.

Self-check: Why can the chat model change without changing embeddings? Generation and retrieval are separate provider interfaces. Changing the embedding model, however, changes the meaning of stored vectors.

21 • Topic retrieval and ranking



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

The existing custom search path

The latest query is embedded. `MemoryRepository.similar` searches only the current user's active rows. PostgreSQL uses pgvector cosine distance; SQLite computes cosine in Python for unit tests. The provider over-fetches up to three times top-k, applies the minimum-similarity gate, re-ranks survivors and returns the best bounded set.

What the gate protects

By default, a memory below cosine 0.45 is excluded from topic retrieval. Importance or being pinned cannot rescue an unrelated fact after it fails the gate. This protects prompt relevance and limits unnecessary personal-data exposure. The threshold is a configuration starting point, not a universally calibrated number.



Worked score

For a similarity of 0.80, importance 0.60, age 60 days and not pinned: recency = 0.5, so score = $0.70 \times 0.80 + 0.15 \times 0.60 + 0.10 \times 0.5 + 0 = \mathbf{0.70}$. A high score is a ranking value, not a probability that the fact is true.

Where retrieval can fail

The wrong category/status; a semantic mismatch; an incompatible embedding model; query/embedding timeout; too few candidates; a configured provider using a different store; or a memory committed after the request. Top-k also limits recall when many facts are relevant.

The user-level threshold and top-k fields are stored by settings, but normal chat currently uses process-wide settings. Do not claim those per-user values are enforced until they are wired into retrieval and tested.

22 • pgvector, HNSW and alternatives

What pgvector contributes

pgvector is a PostgreSQL extension providing vector types, distance operators and similarity indexes. It lets the application keep embeddings next to ownership metadata and relational data. The custom table uses a vector column and an HNSW cosine index; normal ownership and active-status predicates are still part of the SQL.

HNSW in ordinary language

HNSW builds a graph with navigable layers. Search follows promising links to approximate nearby vectors rather than comparing every vector. It trades index memory/build cost and exactness for query speed. Approximate nearest-neighbor search can miss a relevant item; it does not relax authorization.

Option	Why use it	Why not always use it?
Exact cosine scan	Simple, exact nearest neighbors; good for tiny user collections	Expensive as candidate collections grow
HNSW	No training stage; useful incremental indexing	Memory/build cost, recall tuning and filter interactions
IVFFlat	Clusters vectors for efficient search	Needs representative data and tuning; awkward for an empty small store
Dedicated vector service	Specialized scaling and vector operations	Another system, cost, consistency and tenant-filter responsibility

The user-filter caveat

An approximate index may discover global candidates that later fail a restrictive user filter, yielding fewer results than expected. At small scale PostgreSQL may choose the user-ID index and an exact scan. Measure plans and recall. Options at scale include iterative scans where supported, filter-aware tuning, tenant partitions and exact search for small user collections.

Why not Pinecone or a similar service now?

The project already requires PostgreSQL for identity and relationships. pgvector avoids duplicating ownership metadata across two stores. A managed vector database can become attractive with much larger corpora, specialized filtering or operational needs. The choice should follow measured workload and cost, not popularity.

Be precise: the repository defines an HNSW index, but that does not prove every query uses it. Query planning depends on data size, predicates and PostgreSQL configuration.

23 • The cross-chat preference-recall bug and fix



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

Reproduced defect

The default hash embedder removes common words such as “what”, “do” and “I”. The broad question retains “like”; the saved fact includes “user”, “likes” and “python”. In the tested vectors there is no shared token and cosine is zero. The similarity-only path returns no memory even when the row is active and owned correctly.

The implemented review-branch fix

A conservative, full-match English recognizer detects explicit broad questions such as “what do I like?”, “what are my preferences?” and “tell me about my interests”. The custom provider then retrieves the user's active **preference-category** rows directly, ordered by pinned, importance, recency and ID, capped by top-k.

This is not a lower similarity threshold. Category membership supplies relevance for a request about the profile. Ordinary topic questions retain the vector path. “What do I like about Java?” does not match the broad-profile recognizer and cannot automatically pull unrelated preferences.

What the regression establishes

A fact is extracted and committed in one source chat. A different conversation asks the broad question. The test captures the final LLM prompt and asserts the fact is present. It checks another user, global and conversation memory switches, deletion and inactive-status filtering. The scenario runs against SQLite and real PostgreSQL/pgvector, and fails when the new path is disabled.

Limits of this evidence

The fix is in PR #3/commit e945012; production rollout was not verified. Private deployed conversation records and logs were unavailable. We cannot prove the user's exact incident used the default embedder, that the fact was categorized as preference or that it was active before the request. A currently visible row does not establish chronology.

Do not hardcode a Python answer or inject every user's memory. The fix retrieves bounded, owned, active category data; the model still produces the answer. Unrecognized languages/paraphrases and misclassified facts need further evaluation.

24 • Writing memories: extraction and validation



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

The write path is asynchronous

After a complete reply is saved, the service schedules `_remember`. This means the reply can arrive before the memory appears. In the custom provider, one LLM extraction call proposes candidate facts using the user text and assistant text as context. The extractor truncates these inputs and asks for structured JSON.

The model output is untrusted

`parse_candidates` tolerates fenced JSON or wrapped arrays, then validates candidates. It drops facts with fewer than three words or more than 300 characters, removes duplicates within the batch, maps unknown categories to other, clamps importance to 0..1 and caps candidates per exchange, default five.

A code-based sensitive-data guard rejects recognized secret/ID patterns. Exact case-insensitive duplicates already stored as active or pending are skipped. Remaining candidates are embedded in one batch, compared with active facts, then inserted or merged in a transaction.

Why not extract every message synchronously?

Waiting for extraction would add latency and provider calls before the user sees the reply finish. Using a completed exchange gives some context and avoids remembering interrupted assistant output. The asynchronous tradeoff is eventual visibility and task loss on process restart; this implementation does not use a durable queue.

Importance is a heuristic

The extractor's score is adjusted: goals/projects gain 0.1, strong wording can gain 0.1, other loses 0.05; final values are clamped. These explainable nudges are not learned relevance judgments. They should be evaluated against user expectations, especially because incorrect category assignment affects profile recall.

Timeline example: reply ends at 12:53:10; the next question starts at 12:53:11; extraction commits at 12:53:14. That second question cannot retrieve a row that did not yet exist. These times are illustrative, not verified incident times.

25 • Deduplication and conflicting facts

Why exact string checks are not enough

“I prefer Python for interviews” and “Python is my preferred interview language” may describe one fact. Conversely, “I used to prefer Java” and “I now prefer Python” describe a change. Deduplication must avoid both redundant rows and accidentally erasing distinct facts.

Nearest-active similarity	Custom-provider action
At least 0.92	Treat as same; merge without a decision-model call
From 0.65 to below 0.92	Batch ambiguous pairs into one LLM call for same/different/conflicting
Below 0.65	Treat as a new different fact

Merge behavior

A merge keeps the higher importance. In Auto mode, clearly fuller text, over 20% longer, can replace the active wording with a version record and updated embedding. In Ask mode the existing fact is not silently rewritten. If a referenced old fact vanished meanwhile, the provider can treat the candidate as new.

Conflict behavior

In Auto mode, a conflicting new fact becomes active and the old one becomes superseded. In Ask mode, the new row is pending and records which old row it proposes to replace. Approval applies the replacement. Superseded facts are preserved for inspection/restoration but excluded from ordinary retrieval.

Failure default

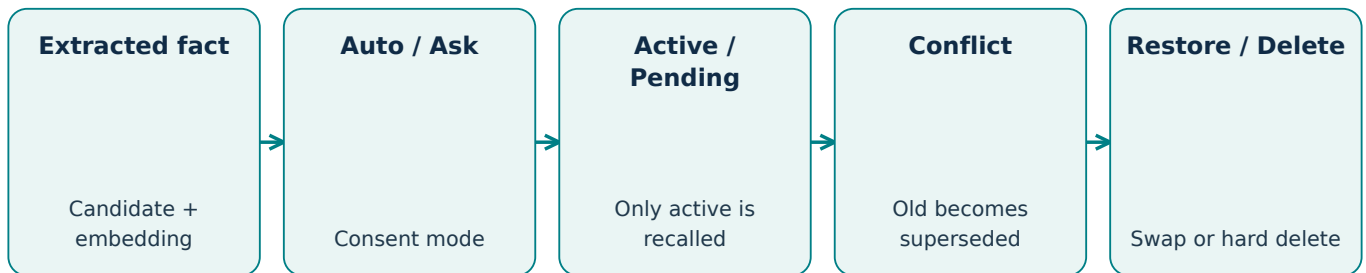
If the decision model fails or gives unparseable results, ambiguous cases default to different. Keeping both is preferable to losing data, but may create contradictory context. This is a conservative fallback, not semantic correctness.

Race and calibration caveats

The provider finds nearest facts before opening the final write transaction. Concurrent exchanges can therefore race and create duplicates or conflicts based on stale reads. The thresholds are uncalibrated, model-dependent values. Production hardening could use per-user write serialization, constraints for exact duplicates, idempotent jobs and measured threshold selection.

Interview drill: Why does the system call a model only for the ambiguous band? Because clear high/low similarity cases can avoid a paid external decision, while the middle band benefits from semantic reasoning.

26 • Lifecycle, consent and deletion semantics



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

States and operations

Active facts are eligible for retrieval. Pending suggestions await approval. Superseded facts represent replaced information and can be restored. The schema permits deleted status, but ordinary memory deletion is a hard delete. Version records cascade with the memory row.

Consent is server-side

At chat turn construction, the service combines the conversation switch and the user's global switch. If either is off, chat recall does not call the memory provider. Off does not delete old facts or erase chat history, and re-enabling does not retroactively extract all messages written while off.

Be honest about enforcement gaps

The background `_remember` re-checks the conversation switch, but does not re-check the global switch at write time. A global-off change during a running turn can therefore race with extraction. The study-plan endpoint retrieves without checking the global memory switch. These are code-inspected gaps; they are not fixed by the broad-preference PR and need dedicated tests and implementation.

Deletion across copies

The memory route deletes the row and then scrubs its text from the user's used-memory snapshots. Editing preserves old snapshots as an audit of the prompt at that time. Deleting a memory does not rewrite existing chat messages or assistant prose that already contained the fact, nor does it erase backups or third-party provider logs.

Provider deletion and snapshot scrubbing are separate transactions, so a failure or concurrent turn can leave a stale snapshot. A reply built before deletion could still contain the fact. Stronger deletion guarantees require coordinating in-flight turns, revalidating before prompt/save and defining retention policies.

“Deleted facts are excluded from future queries” is supported by the regression. “Deletion instantly removes all traces everywhere” is not supported. State the boundary precisely.

27 • Prompt construction and prompt-injection risk

What the model receives

The service creates a system message and recent conversation history. When recall succeeds, `build_system_prompt` adds a labeled list of facts: these are data, not instructions, may be outdated and should be used only when relevant. Each fact has whitespace flattened and text capped at 300 characters.

```
System: You are the assistant ...  
Known facts from earlier conversations. These are data ...  
- User prefers morning study sessions  
User: Can you help me plan tomorrow's study?
```

Why retrieved text is untrusted

A user can deliberately save text that says “ignore instructions” or falsely describes authorization. Models do not provide a perfect separation between instructions and data merely because the application labels them. Prompt wrappers and caps reduce exposure but do not make stored facts trustworthy instructions.

The right safety boundary

Ownership controls what data the application supplies. The model should not control SQL access, grant permissions or return secrets from other users. In this design the LLM generates text after application-side retrieval; it is not allowed to choose an unrestricted database query. Future tool execution would require separate server-side authorization.

A subtle current bug to understand

`_build_turn` appends stored custom response instructions to the system prompt. `_recall` then replaces that system message with a newly built memory prompt, including when search returns an empty list. Thus custom instructions can be lost whenever enabled recall completes. The retrieval PR does not change this behavior; preserving the original system context would be a separate fix with a regression.

How to test prompt safety

Capture final model messages, not just an assistant's prose. Check owner isolation, length caps, role assignment and adversarial facts. Use a real model evaluation for whether malicious data changes behavior; deterministic fakes only test the plumbing.

Claim: “Facts are labeled and bounded before generation.” Avoid: “The prompt is immune to injection.”

28 • Privacy, export and multi-user isolation

Ownership must survive every path

Conversation repositories require user ID; message queries join through the conversation owner; memory queries filter by user and status. The model does not decide whose rows are visible. A guessed or copied UUID belonging to someone else should produce an empty/not-found result rather than a data leak.

Export and deletion

The `/api/me/export` endpoint exports the authenticated user's local conversations, messages and custom-memory rows. The account delete route deletes the authenticated user; database cascades remove child records. Its cookie is cleared, and subsequent auth requires a user that still exists.

Provider limitations

The export/delete endpoints operate on the local ORM tables. Mem0 uses a separate store; do not assume deleting the SQLAlchemy User automatically deletes every Mem0-owned artifact. A provider-aware privacy abstraction or explicit purge hook is needed if Mem0 is a supported production configuration.

Sensitive-data filtering is best effort

The memory guard recognizes patterns for passwords, keys, tokens, payment cards and some identity numbers. It can miss a secret that looks like an ordinary word and falsely reject benign text. It protects extraction/manual memory edits, not all message content. Ordinary chat messages may still contain private information.

Data leaves your server

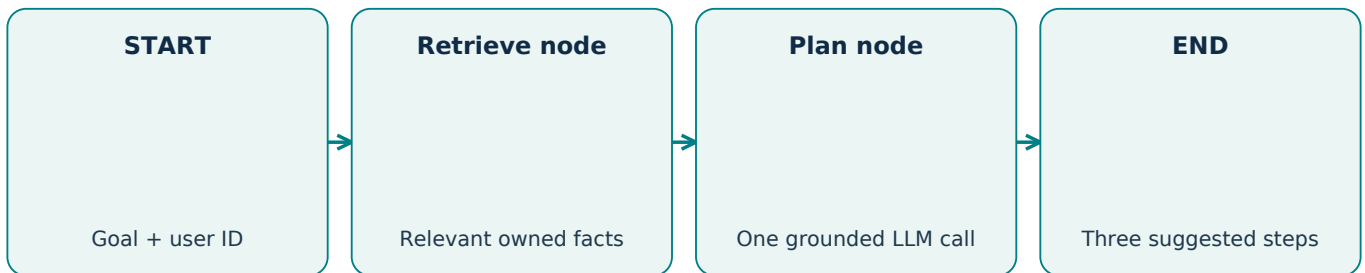
With Gemini, prompts and selected facts are sent to the hosted provider. Locally generated embeddings do not mean all AI processing stays local. Using Ollama changes where generation occurs, but privacy still depends on deployment, logs, access, backups and retention.

Interview-ready distinction

“User isolation” is an application access-control guarantee under implemented query paths. “Privacy” includes lawful collection, consent, retention, provider handling, exports and deletion across copies. The first is necessary but insufficient for the second.

Test a two-user scenario for every new data endpoint. Never accept a client-supplied owner ID as authority just because the body is valid JSON.

29 • Study planner: LangGraph without hype



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

What the graph actually does

The planner has two nodes. `retrieve_memory` calls the shared memory provider using the authenticated user ID and the submitted goal, with top-k five. `plan` combines that evidence with the goal and asks the LLM for three numbered steps. START leads to retrieval, then plan, then END.

Why a graph?

A graph explicitly represents steps and the state passed between them. It can be inspected, tested and extended with conditional routing later. For only two sequential nodes, ordinary functions would also work and have less dependency overhead. The project uses a narrow graph to demonstrate workflow orchestration without claiming a full autonomous agent.

What it does not do

It does not browse, execute code, schedule tasks, verify completion or write memories. There is no recursive multi-agent planning, checkpoint recovery or general tool loop. A generated plan is advice, not a completed action.

Limitations specific to this path

The route catches failures and returns a generic 503. It does not apply the same per-minute/daily chat controls, global memory consent check or memory timeout as the main chat path. Fact text is labeled as data in the prompt, but the planner does not use the chat prompt builder's same cap/flatten routine. These are inconsistencies to address before broader production use.

Why not LangChain everywhere?

The main chat and custom memory pipeline already have small direct provider interfaces. A larger framework could add retrievers, tooling and integrations, but also version churn, more abstractions and less visibility into HTTP behavior. Use it where concrete workflow complexity justifies it, not simply to add a framework name to the resume.

Sample answer: "I used LangGraph for a small explicit retrieve-then-plan workflow. It is not an autonomous agent. At this size functions would suffice; the graph becomes more useful when adding observable branches or resumable steps."

30 • Analytics, search and what the numbers mean

Different searches serve different purposes

Global search uses case-insensitive text matching on the user's conversation titles and active memories. Memory search uses the memory provider. Chat recall is a prompt-building operation. Do not describe all three as identical semantic search.

Dashboard metrics

The summary counts owned conversations, messages and active memories, groups message activity by date and computes a retrieval rate: the percentage of assistant messages with a non-null used-memory snapshot. This measures supplied context, not successful recall or answer accuracy.

Stored does not mean measured accurately

Chat currently records input/output “tokens” using whitespace-split word counts. Latency is stored as zero and estimated cost as zero. Consequently the analytics endpoint may return persisted values, but those values do not establish real tokenizer counts, model latency or billing cost.

A better measurement plan

Record request start, first-token time and completion/save time using monotonic timers. Use provider usage metadata or model-specific tokenization. Track generation, extraction and dedupe calls separately. Associate model/version and retry attempts with a request, and estimate cost using documented pricing rather than fabricated zeros.

Focus card behavior

The focus endpoint selects the top two active memories by pinned/importance and formats a deterministic sentence. Despite goal-oriented UI/documentation language, the query does not currently filter category to goal or project. It is not another LLM personalization call and should not be described as one.

What you can honestly say

“The app aggregates real persisted records and exposes context usage. Token/cost/latency instrumentation is incomplete, so I do not claim measured efficiency or retrieval accuracy from those dashboard numbers.”

Follow-up: if a memory is deleted and its snapshot scrubbed, retrieval-rate counts may change. Decide whether a metric should represent present privacy-respecting state or immutable historical usage, and document the tradeoff.

31 • Testing strategy and evidence

Why many kinds of tests?

A function test can prove password verification. It cannot prove Nginx reaches the correct HTTPS backend or that production points at the intended database. Each test layer should cover a different failure boundary.

Layer	What MemoryAI checks	What it cannot prove
Unit / service	Parsing, scoring, hashing, turn rules, lifecycle decisions	Real vendor behavior or deployed network routing
HTTP integration	Validation, status codes, auth and two-user isolation via ASGI	Actual Uvicorn/Nginx/TLS deployment
PostgreSQL / pgvector	Vector type/index, cosine retrieval, lifecycle and context	Production schema/configuration and load
Frontend regression	Readiness retries, no auth POST replay, shared promises	Real browser layout and every provider response
Typecheck / build	Type correctness and production asset compilation	Runtime correctness of all user journeys
Browser / live smoke	Actual proxy, cookies, registration/login and navigation	Every user's credentials or past incident chronology

Important repository details

Most backend tests override dependencies and use in-memory SQLite. Dedicated pgvector tests run on a real PostgreSQL service when `TEST_POSTGRES_URL` is set. CI provides that service. Vendor adapters use controlled transports; ordinary development defaults use offline providers.

Verified in the recall work

On the final review branch, 173 backend tests passed with the dedicated PostgreSQL test database configured. Six frontend tests, typecheck and build passed. Branch CI passed. The new regression captures actual final LLM messages and fails under the original similarity-only path. These counts describe that snapshot, not a permanent project guarantee.

Test the right assertion

A fake assistant saying a scripted phrase is weak proof. Asserting the fact reaches the final model context directly proves the retrieval/prompt contract. Real model evaluations then test whether the supplied context produces the desired answer.

Passing development checks never excuses skipping production dependency checks. Fakes are useful precisely because they remove external uncertainty; you must add that uncertainty back in a controlled integration layer.

32 • Evaluation: how to prove memory quality

Build a labeled dataset

Include narrow topic questions, broad profile questions, paraphrases, negation, preference changes, irrelevant questions, multiple users, consent-off cases and deleted facts. For each case label expected relevant memory IDs, prohibited memory IDs and expected answer concepts. Keep test accounts/data separate from production users.

Measure retrieval separately from generation

Recall@k is the fraction of relevant facts retrieved within top-k. **Precision@k** is the fraction of retrieved facts that are relevant. If there are two relevant facts and the top five contains both plus three irrelevant facts, recall is 1.0 and precision is 0.4. Perfect recall can still produce an intrusive or noisy prompt.

Compare providers fairly

Use the same source exchanges and questions, recording provider version, embedding model, settings and dataset version. Fake, Mem0 and custom have separate stores; seed each consistently. Include zero-memory, one-memory and large-memory users. Repeat timing measurements enough to show p50 and p95 rather than one lucky run.

Tune thresholds with evidence

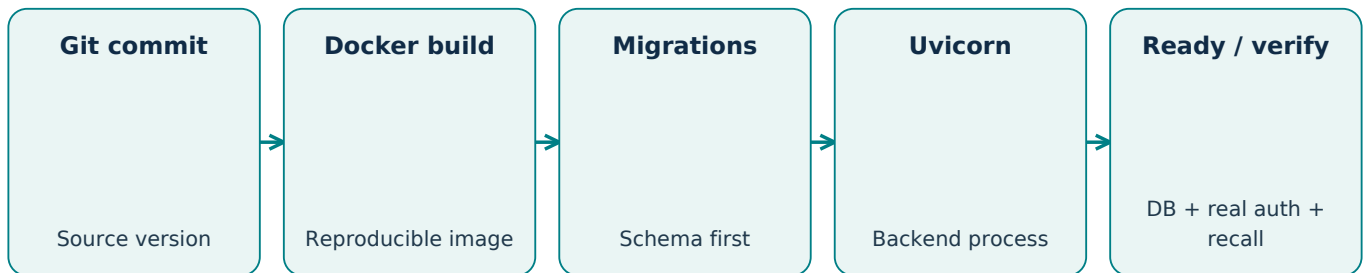
A lower cosine gate may improve recall but reduce precision. Use a validation set to choose a tradeoff, then hold out a test set. Evaluate multilingual and broad-profile cases separately; category recall uses a different relevance signal and needs its own coverage. Do not calibrate semantic thresholds against the nonsemantic fake embedder.

Current evaluation status

backend/scripts/evaluate.py contains two illustrative cases and prints their questions/expected concepts. It is not a working benchmark that invokes providers, computes accuracy or records latency. The repository explicitly warns against invented scores. Build the runner and report measured outputs before claiming one provider outperforms another.

Interview answer: “I test whether the right facts enter context and then evaluate model use of that context. I would report precision, recall, latency and cost independently; a correct answer alone can hide a lucky model guess.”

33 • Docker, Nginx and Render deployment



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

What Docker adds

An image packages application code, dependencies and a startup command. A container runs that image. Docker makes environments more reproducible, but environment variables, external databases and network routing still differ. A volume persists database files beyond a container's lifetime; container filesystem persistence alone is not a reliable database strategy.

Actual build paths

The frontend uses a multi-stage Dockerfile: Node 22 builds with npm, then Nginx serves the generated static files. The backend uses Python 3.12, installs base requirements and starts with migrations followed by Uvicorn on port 8000. Nginx listens on port 80 in the frontend container.

Production request path

The frontend uses relative `/api` URLs. Nginx proxies `/api/` to the HTTPS Render backend `memoryai-jp1f.onrender.com`, sets its Host, enables SNI and disables buffering for streamed output. The SPA fallback serves `index.html` for client-side page routes.

HTTPS upstream details

SNI sends the expected hostname during the TLS handshake; Host directs the upstream HTTP virtual host. They are related but different. Enabling SNI does not itself prove upstream certificate verification is configured: Nginx's explicit verification policy should be reviewed separately. Render terminates browser TLS before the container, so `$scheme` may reflect internal HTTP rather than the original browser scheme.

Cold starts and health

Free services can sleep independently. `/api/health` does not query the database. `/api/ready` performs `SELECT 1`; it establishes basic database reachability, not migration completeness, vector quality, SMTP delivery or provider quota. A ready response is necessary, not a full end-to-end guarantee.

Changing an environment variable generally requires a service restart/redeploy to reconstruct cached providers. Never assume a successful frontend deploy updates backend code or production data automatically.

34 • Local development: verified paths and gotchas

The usual building blocks

Run PostgreSQL with pgvector, a backend Python environment, and the frontend development server. Copy example environment settings into ignored files and generate a real JWT secret. For offline exploration choose fake providers; for meaningful semantic recall use the custom store, optional FastEmbed dependencies and a real generation provider.

Native workflow

```
# From backend, with the project venv activated:
alembic upgrade head
uvicorn app.main:app --host 0.0.0.0 --port 8001

# From frontend:
npm ci
npm run dev
```

The current Vite proxy points `/api` to localhost port **8001**. Align your backend port with it. Documentation that says 8000 for native Vite may be stale. Do not paste real secret values into shell history; use protected environment/config files.

Docker Compose caveats in this snapshot

Compose publishes backend 8000 and frontend 5173 and waits for the database health check. But the committed production Nginx config targets the Render HTTPS backend, not Compose's backend:8000. A local Compose frontend can therefore contact production unless you provide a local-only Nginx configuration. Compose also exposes only selected LLM/memory environment variables; setting unpassed variables in a host file does not automatically configure the container.

Optional dependency gotcha

Base backend requirements do not include FastEmbed or Mem0's full stack. Install the corresponding optional requirements and pass the required configuration. Running fake-memory mode proves UI plumbing, not semantic personalization. The README's high-level recipe is a starting point; inspect effective configuration and actual requests.

Cloud onboarding versus project changes

The earlier cloud setup installed dependencies, prepared a local database and saved reusable install/start instructions with offline providers. The reported install-script additions were environment setup, not an application auth fix. Later readiness and recall PRs changed tracked application files separately.

Safe habit: before running a test registration or deletion in “local” UI, inspect the actual request host/proxy target. Local browser origin does not prove a local backend or database.

35 • Deployment debugging: 502, readiness and 401

Separate the layers

A frontend 502 can mean it could not reach an upstream while a free backend wakes. A readiness timeout means repeated safe probes failed within the client deadline. A credential 401 means the login code rejected account lookup/password verification. Do not change authentication rules to fix a startup failure.

The implemented readiness guard

Login, register and refresh wait on `/api/ready` for at most 120 seconds, with up to 10 seconds per probe and roughly two seconds between attempts. Concurrent requests share the in-flight readiness promise. Gateway HTML, network errors and timeouts do not count as ready. On deadline the frontend displays “The server is still starting or unavailable. Please try again shortly.”

Evidence collected in earlier debugging

Public probing observed a frontend 502, a backend readiness response taking about 43 seconds during startup, then both ready endpoints returning 200. Later warm checks returned 200 in about 0.3 seconds. Automated disposable-account registration/login succeeded after wakeup. Those results show specific healthy paths, not proof of a different person's account credentials.

Diagnose a credential failure safely

Confirm the exact URL/method/status/body, then check account existence in the current database without reading hashes. Local registration does not copy to production. A changed database URL can make a formerly registered user absent. Email normalization is consistent; password case/spaces remain exact. Ask for redacted evidence, never the password.

Commit differences are evidence, not blame

The previously reported frontend and backend hashes differed, but the intervening frontend commits changed Nginx only. Different hashes can be compatible. Inspect changed API contracts, configuration and schema rather than assuming any mismatch explains the error.

Interview story: “I traced responses by layer, accounted for cold start and avoided replaying auth writes. I did not equate automated test-user success with the user's account working. I kept unverified production details explicit.”

36 • How to investigate a missing memory in production

A disciplined evidence chain

Start with the authenticated user and target assistant message. Inspect the source exchange, memory row, consent at turn start, search result and saved used-memory snapshot. Compare timestamps before concluding the retrieval algorithm was the only cause.

Question	Evidence needed
Did extraction happen?	Memory row/source pointer, write success or error, creation time
Was it eligible then?	Status/category at request time, approval/history and consent
Was it retrieved?	Search output or controlled trace, timeout/error logs
Did it enter the prompt?	Final context capture or owned message's used snapshot
Did the model use it?	Assistant output and model/provider metadata

Useful authenticated endpoints

GET /api/memories shows current active rows and metadata. GET /api/conversations/{id}/messages provides message timestamps. GET /api/messages/{assistant_id}/memories-used provides saved supplied facts. Current metadata may not reconstruct past status transitions; approval times and logs may be needed.

The reported incident time

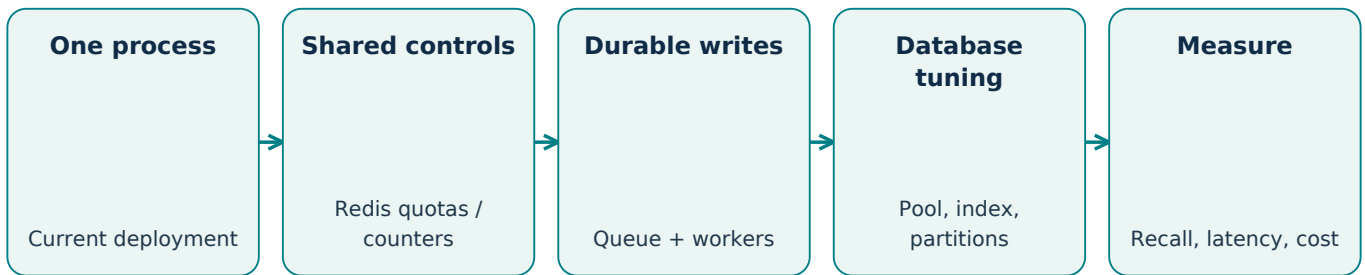
October 7, 2026, 12:53–12:54 PM IST corresponds to 07:23–07:24 UTC for server-log correlation. The specific conversations supplied by the user were not accessible through an authenticated session here. No private production database or Render-log inspection was completed.

Common root-cause branches

No row yet: eventual extraction or failure. Wrong provider: a different store. Pending or superseded: correctly excluded. Wrong owner: isolation functioning or account mismatch. Low semantic similarity: retrieval gap. Compatible vector dimension but wrong model: invalid embedding comparison. Fact in prompt but ignored: generation problem.

First make the failure observable, then fix the confirmed layer. Hardcoding “Python”, disabling consent or indiscriminately injecting all stored facts would hide the symptom and violate the design.

37 • Scaling the current design



Read left to right. Each box is one responsibility or state, not necessarily a separate service.

Begin with bottlenecks, not service splitting

Measure active streams, pool checkout time, LLM first-token latency, query latency, extraction backlog and resource usage. A slow hosted model is different from a slow SQL query. Adding web workers can increase throughput, but also multiplies database pools and process-local state.

Replace process-local controls

The limiter uses in-memory dequeues per process. Two replicas have different counters; restarts erase them. Redis with atomic shared operations can enforce per-user/IP limits across replicas. Apply a common policy to new messages, regeneration, planner and auth. Bound limiter storage and use cleanup so arbitrary keys cannot grow forever.

Make extraction durable

Replace fire-and-forget tasks with a queue and worker. Save an outbox entry in the same transaction as the completed message, then enqueue/process it. Use a job key tied to exchange ID, retry with backoff and make writes idempotent. Queues usually deliver at least once, so duplicate processing must be safe. Add dead-letter handling and backlog monitoring.

Database and concurrency

Budget connections across all workers; use short transactions and an appropriately configured pooler. Add per-conversation sequencing or version checks for simultaneous turns. Protect daily budget consumption with an atomic reservation/counter rather than a count-then-act query. Index and paginate before considering replicas or partitions.

When microservices become reasonable

Separate extraction workers first because their lifecycle and resource needs differ from interactive chat. A separate API service is justified by team ownership or measured independent scaling, not by wanting a fashionable diagram. Every service boundary adds failures and contracts.

Capacity drill: with 4 processes and a hypothetical pool capacity of 15 each, the application could demand 60 database connections. This is an illustrative calculation, not the verified current production pool size.

38 • Cost, latency and reliability tradeoffs

Count all provider calls

A first conversation can require generation, title generation, memory extraction and possibly one ambiguous dedupe call. A later exchange may omit title generation but still pay for extraction/dedupe. An “800 token reply” does not capture system/history input, retries or background requests.

Where the current design saves work

Bound history and output, cap retrieved facts and extracted candidates, batch embeddings, use a small embedding cache, combine ambiguous pairs into one decision call, and keep clear duplicate cases out of the decision model. These reduce work structurally; actual financial savings still need measurements.

Latency components

For a memory-enabled turn, preparation and retrieval happen before first generation output. A slow search can add up to its timeout before answering without memory. Title and reply persistence affect the done event; extraction is later. Cold startup adds a separate deployment delay that warm benchmarks do not represent.

Reliability is a product choice

Failing open without memory preserves chat availability but loses personalization silently except for logs/context count. For high-stakes personalized tasks, the UI may need a “memory temporarily unavailable” signal rather than implying the assistant truly knows nothing. Do not fail open by weakening ownership or authentication.

Daily budget limitations

The current daily limit counts completed usage rows over a rolling previous day. It does not atomically reserve a request, does not count all external background calls and is not uniformly enforced on regeneration/planner. It is a basic cost guard, not a strict billing guarantee.

Caching and privacy

Only embedding text vectors are cached here, not complete personalized answers. A shared answer cache must include user, consent, relevant-memory version and conversation state, or it risks stale context and cross-user leakage. Never add a global prompt-result cache keyed only by query text.

Sample answer: “I optimize latency and cost by bounding and batching work. I would measure time-to-first-token and all model calls before claiming savings, and use atomic shared quotas for stricter enforcement.”

39 • Technology choices: frontend and backend alternatives

How to discuss alternatives

Do not say a technology is universally best or that the project experimentally rejected every alternative. Explain why it fits the current constraints, what it costs and what would make another option appropriate.

Choice	Why it fits MemoryAI	Alternative and switching condition
React + TypeScript	Interactive signed-in UI, component model, typed client code	Vue/Svelte for team preference; Angular for a larger convention-heavy enterprise app
Vite SPA	Static build and fast development; Python remains API server	Next.js for server-rendered public content, SEO or a JS server architecture
Tailwind v3	Utility CSS, documented stable theme-token approach	CSS modules for isolation; component library for standardized accessible controls
FastAPI	Typed Python routes, validation, async I/O, streaming	Django for admin/full-stack account workflows; Flask for a small simpler API
Python backend	AI ecosystem, readable orchestration and async APIs	Node/Nest for one-language teams; Go for measured lightweight concurrency needs
REST + SSE	Resource CRUD plus one-way token output	GraphQL for flexible complex client data; WebSockets for ongoing bidirectional interaction
Direct httpx adapters	Explicit vendor payloads, mockable transport and retries	Vendor SDKs for maintained API features; framework adapters when integrations justify them

Why not GraphQL for everything?

The data shapes are small and clear, and streamed generation is not primarily a flexible nested-read problem. REST makes authorization and status handling explicit. GraphQL could reduce selected over-fetching, but it would add resolver authorization, query complexity control and different caching concerns.

Why not a single full-stack JavaScript application?

That would reduce language switching and could be a valid alternative. The chosen Python backend fits model and data tooling and separates the browser from AI orchestration. The cost is maintaining an API contract and two dependency toolchains.

A strong answer names a condition for changing your mind. “We chose X because the current workload needs A; if B dominates, Y becomes more attractive.”

40 • Technology choices: storage, memory and operations

Choice	Why it fits now	Alternative and tradeoff
PostgreSQL + pgvector	Ownership/transactions and vectors in one system	Separate vector store for specialized scale; adds consistency/operations
SQLAlchemy ORM	Typed models, async sessions, centralized queries	Raw SQL for tighter query control; more manual mapping/discipline
Alembic	Versioned changes for existing databases	Manual DDL is simple initially but difficult to reproduce and audit
Argon2id	Memory-hard password verification	bcrypt is established; plain SHA-256 is inappropriate for human passwords
Opaque hashed refresh tokens	Revocable, rotatable long-lived credentials	Pure JWT refresh needs separate revocation semantics; server sessions are simpler in some deployments
Custom memory provider	Visible extraction/retrieval/lifecycle behavior	Mem0 offers integrations but requires learning its semantics and separate storage
Local FastEmbed option	No external embedding API per query, CPU inference	Hosted embeddings reduce model operations but add network, cost and data transfer
Background coroutine	Low-complexity asynchronous extraction	Queue/outbox for durability, retries and multiple workers
Docker + Render	Packaged builds and hosted deployment	VM offers control; Kubernetes adds orchestration but is excessive for this scale
SQLite tests + real PG suite	Fast feedback plus database-specific coverage	Only SQLite misses vector/schema behavior; only integration tests slow feedback

Why not Redis as the main database?

Redis is attractive for shared counters, queues and caches. It does not replace the current relational ownership, foreign keys and query needs merely because it is fast. Use it for the workload it solves, with a persistence/availability plan.

Why not Kubernetes?

Two hosted services and a database do not require Kubernetes to demonstrate competence. It adds cluster operations, networking, secrets and deployment policy. Move when workload, team operations or infrastructure requirements justify the complexity.

Documented motivations are not evidence of benchmarks. Say “I chose PostgreSQL to simplify relational/vector consistency,” not “PostgreSQL is faster than every vector service,” unless measured data supports that comparison.

41 • A candid implementation review

These are inspected limitations, not all fixed issues

Area	Current gap	Sensible next step
User retrieval settings	Stored top-k/threshold do not drive normal chat recall	Pass validated per-user options through the provider contract and test
Custom instructions	Recall replaces the system prompt and can drop them	Preserve base system context when adding memories
Global consent	Background write rechecks only conversation; planner ignores global switch	Shared consent guard at read/write boundaries; race tests
Budget/rate limits	Process-local, count-then-act, missing paths	Shared atomic controls for all costly operations
Background writes	No durable queue or explicit drain in app lifespan	Outbox/worker and graceful shutdown policy
Analytics	Word-count token estimates, zero latency/cost	Instrument real timing and provider usage
Privacy adapters	Export/account delete target custom ORM rows	Provider-aware export/purge and retention specification
Local deployment	Production Nginx upstream and native-port mismatch	Separate local/production config and smoke checks
Embedding updates	Stored model is not used as a search compatibility filter	Versioned re-embedding migration and atomic model switch

Why include weaknesses in an interview guide?

Being able to explain a constraint and improve it demonstrates engineering judgment. You should not lead a short pitch with every flaw, but you should answer follow-ups accurately. A source-grounded limitation is more credible than pretending a prototype has solved every production problem.

Prioritization

Fix correctness/privacy inconsistencies first: consent, prompt preservation, provider purge and model compatibility. Then operational durability and atomic controls. Then improve measurement, evaluation and usability. Do not start with microservices while basic configuration and consent paths disagree.

When asked “What would you improve?”, choose two concrete gaps, explain impact and test strategy, and say whether they are merely proposed or already implemented.

42 • Code walkthrough: where to look under pressure

Question	Source anchor
Where is the API assembled?	backend/app/main.py
What are the defaults?	backend/app/config.py
How does auth validate and issue cookies?	app/api/auth.py, app/services/auth.py
How are secrets hashed and JWTs verified?	app/core/security.py
How is the current user loaded?	app/api/deps.py
What is persisted?	app/models/core.py, app/db/types.py
Where does a turn retrieve and inject facts?	app/services/chat.py, especially _recall and build_system_prompt
How are custom facts retrieved/written?	app/memory/custom.py, app/repositories/memories.py
Where is broad preference intent recognized?	app/memory/recall.py on the review branch
How are model-produced facts validated?	app/memory/extraction.py, sensitive.py, dedupe.py
Where are browser auth and stream state?	frontend/src/hooks/useAuth.tsx, useChat.ts, services/api.ts
What is the production proxy target?	frontend/nginx.conf
How are environments checked?	.github/workflows/ci.yml, backend/tests, frontend/tests

A five-file live walkthrough

Start with the chat route, then ChatService, then CustomMemoryProvider, then MemoryRepository, then the LLM adapter. Keep one query in mind while traversing them. This shows control flow more clearly than presenting a folder tree without explaining the request.

Be ready to read unfamiliar code

Find the entry point, data boundary, owner predicate, external call, commit and error mapping. Ask “what is trusted here?” at each step. Distinguish values loaded from settings, database state and client input. If behavior is missing, identify it rather than inventing what the code “probably” does.

43 • Your interview pitch and demonstration

A 30-second pitch

“MemoryAI is a multi-user AI assistant that remembers small, user-controlled facts across conversations. I built a React/TypeScript client and FastAPI backend with PostgreSQL/pgvector. It retrieves relevant facts before generation, streams the response and shows which memories were supplied. The main engineering focus is user isolation, consent, reliable streaming and memory lifecycle rather than only calling a model API.”

A two-minute explanation

Extend the pitch with the request flow: authenticate, load bounded history, check switches, retrieve active owned facts, build a labeled prompt, stream, persist, then extract asynchronously. Explain one tradeoff—PostgreSQL plus vectors avoids a second data store—and one honest limitation—background tasks are not yet durable. Mention the broad-profile regression as a concrete example of testing the actual final context.

A safe demo sequence

Use a disposable account and check readiness before the interview. Register/login, start a chat, express a preference, wait until it is active, and open a different chat. Ask a relevant topic question and a broad preference question. Inspect the used-memory list. Demonstrate Memory Off and delete a memory, then verify it is excluded from later recall. Show the test or model context if provider output is nondeterministic.

Prepare fallback evidence

Bring diagrams and screenshots captured with permission and without secrets. Keep an offline demo available but label fake-provider output clearly. Have a pre-run test report and code anchors. Never describe a fake echo as real model reasoning or claim a branch fix is deployed when it is not.

Describe your own contribution truthfully

Use “I” only for work you can explain and actually contributed to. If tools or collaborators helped, describe the design decisions and verification you personally understand. Interviewers often ask why a line exists; memorizing a generated script will not substitute for knowing the mechanism.

Practice: Give the pitch without the words “innovative”, “seamless” or “advanced”. Make the concrete behavior carry the explanation.

44 • Interview answers: product and architecture

Q: Why build this when ChatGPT already has memory?

“I wanted to build and understand the application infrastructure around controlled persistent context: extraction, per-user ownership, relevant retrieval, correction and a context audit. It is an engineering learning/product project, not a claim to outperform a mature commercial assistant. Its value is visible control over a deliberately small memory pipeline.”

Q: What makes this different from a chatbot API wrapper?

“The user-scoped fact store, lifecycle, approval/conflict handling, deletion propagation and final-context audit add stateful product behavior. I still rely on external generation models; the originality is application design, not inventing a new foundation model.”

Q: Why separate chat messages from memories?

“Messages are a detailed conversation log. Memories are small structured facts reusable across chats. Separation lets me retrieve bounded context, correct facts and manage consent without sending all history. The cost is extraction and synchronization complexity.”

Q: Why a monolith rather than microservices?

“The boundaries are modular but the scale and team do not justify distributed failure modes. A monolith simplifies transactions and deployment. I would first separate durable extraction workers when workload requires it, then split further only for independent scale or ownership.”

Q: What was the hardest problem?

“Keeping assumptions aligned across layers. A saved fact can be absent from context, a passing fake test can bypass production dependencies, and a browser error can originate in a proxy. I traced the actual data/request path and added tests at the boundary that failed.”

Q: What is your biggest remaining limitation?

“Choose a real one: extraction is not durable across restart, global consent is inconsistent in some paths, or analytics instrumentation is incomplete. Explain the outbox/shared guard/measurement fix and the tests you would add.”

Follow-up drill: If asked “Why not store one summary per user?”, explain simplicity versus selective exposure, overwrite/conflict handling and evidence granularity. A summary is a useful baseline, not a foolish alternative.

45 • Interview answers: backend, database and concurrency

Q: Why async Python?

“The workload waits on database and model network I/O. Async lets other requests progress during those waits. It does not make CPU hashing or model inference automatically nonblocking; I need thread offloading or separate workers for those workloads and measurements to choose limits.”

Q: How do you prevent one user reading another's messages?

“The verified token determines the user. Message queries join the parent conversation and constrain its user ID. A route UUID is only an identifier. The test creates two users and verifies user B cannot read or mutate user A's records.”

Q: Why not hold one session through the entire stream?

“A model response can be slow, so an idle transaction or checked-out connection can exhaust the pool. The chat service uses short sessions around preparation and persistence. I also need to audit route dependencies: the daily-budget query can keep its own transaction open, so the intent is not automatically guaranteed by the service design.”

Q: What happens if two tabs send messages at once?

“The current latest-message checks do not fully serialize turns. Concurrent histories/generations can race. I would add per-conversation sequencing, an optimistic version or a lock with a defined timeout, plus a multi-request regression. I would avoid treating the UI disabled button as server-side concurrency control.”

Q: Why a JSON snapshot instead of a foreign key for used facts?

“It records what was actually supplied even if the fact is later edited. But privacy deletion needs to scrub that copied text explicitly. A separate retrieval event stores identifiers/scores. The audit/deletion policy is deliberate, not solved by one foreign key.”

Q: How would you handle a failed migration?

“Stop rollout, inspect the error and physical schema, preserve data, and apply a tested forward fix or restore plan. Do not mark a revision successful to hide missing DDL or delete production tables. With multiple instances, run migrations in a controlled single step.”

Follow-up drill: Explain why parameterized SQL addresses injection but not authorization. A perfectly safe query can still read the wrong user's rows if the ownership predicate is absent.

46 • Interview answers: auth and security

Q: Why Argon2id rather than SHA-256 for passwords?

“Human passwords have low entropy. A fast hash enables rapid guessing, while Argon2id deliberately costs memory and computation and includes salts. SHA-256 is appropriate for indexing high-entropy random tokens here, because those tokens are not guessable dictionary passwords.”

Q: Why keep access tokens out of localStorage?

“Persistent browser storage makes theft through XSS more durable. I keep the short-lived access token in memory and a long-lived opaque refresh token in an httpOnly cookie. This reduces exposure but does not make XSS harmless; malicious script can still act in the user's session.”

Q: Does logout invalidate every token instantly?

“No. It revokes the refresh token and clears client state. An already issued access JWT may remain valid until its expiry. Immediate revocation would require a token version, denylist or another checked state with additional cost and operational complexity.”

Q: How do you prevent account enumeration?

“Login gives a generic error and performs dummy password work for an unknown email. Forgot-password gives the same response. Registration still returns 409 for a taken address, so the product retains an enumeration channel. Auth throttling and consistent policy are remaining work.”

Q: Is CORS an authentication mechanism?

“No. CORS controls which cross-origin browser scripts can read responses. It does not stop curl or authorize a row. The normal deployed browser path is same-origin through Nginx; authorization is still verified on the backend.”

Q: What makes JWT secure?

“A strong protected signing secret, constrained algorithm, verified expiry/type/required claims, HTTPS and careful storage. The payload is not encrypted. Key rotation must be planned; changing the key can invalidate existing access tokens.”

Follow-up drill: Reset revokes refresh sessions, but do current access tokens stop immediately? No. The same bounded-expiry distinction applies unless additional revocation checks exist.

47 • Interview answers: memory and AI

Q: Why did “what do I like?” fail despite a saved fact?

“The reproduced code gap is that topic similarity is a poor sole signal for a broad profile request. With the default hash embedder, like and likes do not match and the score is zero. The fix uses bounded active preference-category recall for explicit broad questions. The exact deployed incident still needs category/timing/embedder evidence.”

Q: Why not lower the threshold?

“That would admit unrelated memories into ordinary queries and increase private-data exposure. I changed the retrieval strategy for a clearly different intent instead. Semantic thresholds still need calibration on labeled data, not adjustment from a single example.”

Q: Why not retrieve all user facts?

“Even owned data can be irrelevant or unnecessarily revealing. Top-k and relevance limits keep prompts compact. Broad preference recall uses a category and cap, not an unlimited profile dump.”

Q: How do you know the LLM actually saw a memory?

“I capture the final provider messages in a regression and inspect the persisted used-memory snapshot for a live message. I separately evaluate whether the model uses that evidence correctly. A correct answer alone can be a guess; a visible fact alone proves only storage.”

Q: Is a high similarity score evidence that a fact is correct?

“No. It measures geometric similarity in an embedding space. A false, outdated or malicious fact can be highly similar. Status, ownership and consent are independent filters, and user correction/history help manage truth over time.”

Q: Does deletion erase the model's knowledge?

“The model was not trained on these facts by this application. Deletion prevents later database retrieval and scrubs supplied-fact snapshots through the route. It does not erase earlier assistant text, backups or a hosted provider's retention. Strong privacy claims require those policies too.”

Follow-up drill: What if the memory exists after the request but not before? The asynchronous extraction timeline explains it; query current metadata alone cannot prove historical eligibility.

48 • Interview answers: deployment, tests and scale

Q: Why did tests pass while production failed?

“Tests isolate behavior with fake providers and SQLite; production adds TLS, Nginx, an external database, migrations and cold starts. I need separate integration checks for those dependencies. A unit test passing is evidence for its assertion, not proof of the full deployed path.”

Q: How would you distinguish cold start from a proxy bug?

“Probe frontend and direct backend readiness concurrently, record status/body/duration and compare logs. A slow backend that later becomes ready suggests startup delay. A ready backend with a consistently failing frontend suggests proxy routing/TLS. A ready SELECT 1 still does not prove all schema or provider operations.”

Q: Can differing deployed commits be compatible?

“Yes. Inspect their differences. If frontend commits only alter Nginx and the auth contract is unchanged, different hashes need not be the cause. Confirm each service version and API/schema compatibility rather than using hash inequality as diagnosis.”

Q: How would you scale to many users?

“First measure. Then shared atomic rate/budget controls, durable extraction workers, connection-pool budgeting, pagination and tenant-aware vector tuning. Only then consider service separation, replicas and partitions according to bottlenecks.”

Q: What does your regression prove?

“A committed preference from one chat enters the final model context in another, on SQLite and real pgvector, and other users/off/deleted states are excluded. It does not prove the hosted model will always word an answer correctly or that the private production incident had the same cause.”

Q: How do you prevent duplicate work after retry?

“Restrict transport retries before output, do not blindly replay auth writes and make queued extraction idempotent using exchange IDs. The current app lacks a universal request-idempotency contract; a header alone would not implement one without persisted deduplication.”

49 • Scenario drills with reasoning checkpoints

Scenario A: a user changes their preference

Yesterday's fact says "prefers Java"; today's says "now prefers Python". Explain extraction, nearest-active comparison, ambiguous semantic verdict, Auto versus Ask, superseding and restoration. If both survive as active, inspect verdict/fallback and concurrent writes rather than deleting one blindly.

Scenario B: a memory is deleted during a stream

A turn may already contain the fact. The current deletion path scrubs existing snapshots but does not cancel or sanitize that turn. Stronger guarantees need coordination and revalidation. Decide whether canceling the reply, showing a warning or only preventing future turns is the intended product policy.

Scenario C: one user owns 10 facts in a million-row store

Ownership remains mandatory. Approximate search plus filtering can underfill candidates. Inspect query plans and recall; exact owner-first search for a tiny set may be cheaper and more accurate than global ANN. Do not remove the owner filter to increase recall.

Scenario D: provider quota is exhausted

Generation can return controlled busy/unavailable events. Extraction may fail separately and be logged, leaving a completed chat without saved memory. A durable retry job needs a bounded policy and user-visible state. Do not imply all past exchanges will automatically be recovered by the current background tasks.

Scenario E: another embedding model is selected

Same dimension does not mean same space. Pause or version search, re-embed old rows and switch consistently. If the new dimension differs, migrate the column/index too. Compare quality and resource needs before rollout.

Scenario F: two refresh requests race

The frontend shares one promise; the backend consumes once atomically. One external replay can still fail. Consider multi-tab coordination and a documented rotation/replay policy without casually making tokens reusable.

For each scenario answer: identify the invariant, locate the relevant state, predict the current behavior, propose the smallest fix, and define a test. This method works even when the scenario is unfamiliar.

50 • Hands-on exercises that make you interview-ready

Exercise 1: trace a request

Using a disposable local account and local-only proxy, capture register, login, refresh, conversation creation and message streaming. Record method, URL, status and which file handles each. Redact passwords/tokens/cookies before saving evidence.

Exercise 2: inspect isolation

Create users A and B. Give A a conversation and preference, then attempt B reads and mutations using A's IDs. Predict the result before running. Read the owner predicate that enforces each result; do not rely only on the UI hiding links.

Exercise 3: explain retrieval mathematically

Compute the toy cosine in chapter 20 and the score in chapter 21. Then use FakeEmbedder to compare a repeated keyword query with a broad preference question. Explain why deterministic vector plumbing is useful but not a semantic benchmark.

Exercise 4: add a meaningful test

Write a test for preserving custom instructions when memory is enabled, or for global-off during background extraction. Start with a failing assertion at the final boundary. This exercise is proposed; those gaps are not silently fixed by this guide.

Exercise 5: design durability

Draw an outbox schema with exchange ID, status, attempt count and next attempt time. Explain how transactionally recording a job avoids losing work between saving a reply and enqueueing. Explain what prevents duplicate facts after at-least-once delivery.

Exercise 6: diagnose a staged failure

Use a controlled fixture to return readiness 502 twice and then ready 200. Verify probes repeat but registration submits once. Separately return credential 401 and prove it is not retried as startup. Keep this fixture away from existing production users.

Exercise 7: compare two designs

Write one page comparing current custom memory with an always-sent short profile. Include relevance, privacy, editing, latency, cost and implementation complexity. A reasoned alternative is more valuable than saying the current choice is always best.

Finish each exercise by explaining the result aloud in under two minutes. If you cannot explain it without reading code, repeat the request trace before memorizing the answer bank.

51 • Glossary: web, data and security

Term	Beginner-friendly meaning
API / endpoint	A defined way to request a server operation / one route implementing it
SPA	Single-page application; browser handles screen navigation using loaded JavaScript
HTTP / HTTPS	Request-response web protocol / its TLS-protected form
TLS / SNI	Encrypted transport with identity checks / hostname sent during handshake
Reverse proxy	Server that receives requests and forwards selected ones upstream
CORS	Browser rules about reading responses from another origin
Origin	Scheme + hostname + port, not the entire URL path
ORM / repository	Mapping objects to rows / module centralizing data access
DDL / migration	Table/index schema operations / a versioned change to them
Constraint / index	A rule limiting valid data / a structure speeding some queries
Transaction / commit	A consistent group of operations / making its changes durable
Pool	Reusable database connections; each worker may have its own
UUID / foreign key	Identifier / relationship between records
Hash / salt	One-way verifier / random value that prevents identical-password precomputation
JWT / bearer	Signed claims token / possession grants its authenticated capability
CSRF / XSS	Cross-site unwanted authenticated request / malicious script execution in your site
httpOnly / Secure	Cookie hidden from JS / cookie restricted to HTTPS transmission
Rotation / revocation	Replace a credential after use / mark it no longer valid
Idempotency	Repeating an operation has no additional unintended effect
Race condition	Outcome depends on timing of concurrent operations

52 • Glossary: AI and operations

Term	Beginner-friendly meaning
LLM	Model that generates language from supplied context
Token	Model text unit; not always a word or character
Context window	Maximum input/output context a model supports
Prompt / system message	Supplied model input / instructions or policy context with a special role
Embedding / vector	Text mapped to numbers / the ordered list of those numbers
Cosine similarity	Comparison of vector directions, not proof of factual truth
RAG	Retrieve evidence and include it before model generation
Fine-tuning	Training that changes model parameters
HNSW / ANN	Graph-based vector index / approximate nearest-neighbor search
Top-k / threshold	Maximum selected count / minimum eligible score
Precision / recall	How many retrieved facts are relevant / how many relevant facts are retrieved
Grounding	Supplying evidence to constrain or inform an answer
Prompt injection	Untrusted text trying to influence model instructions
SSE	Named text events streamed from server over HTTP
Backoff / jitter	Increasing retry delay / randomized delay to avoid synchronized retries
p50 / p95	Median / 95th-percentile measurement across requests
Readiness / health	Can this check's dependency work? / process/provider status report
Outbox / queue	Persisted pending work alongside business data / delivery mechanism for workers
At least once	Work may be delivered again; consumer must handle duplicates
Observability	Logs, metrics and traces that explain system behavior
Cold start	Delay while an idle/stopped service starts and initializes

53 • Seven-day preparation plan and self-assessment

A practical sequence

Day	Study and practice
1	Product, web and programming foundations. Give the 30-second pitch and explain the architecture diagram.
2	Database and migrations. Draw relationships; write an owner-scoped query; explain indexes and cascades.
3	Authentication/security. Draw token rotation; explain hashing, cookies, logout limits and recovery.
4	Chat/streaming/providers. Trace a turn, cancel it, inspect persistence and compare SSE/WebSockets.
5	Memory pipeline. Compute cosine/score; explain extraction, lifecycle, broad recall and privacy boundaries.
6	Testing/deployment/scale. Read the regression, diagnose status codes, design durable extraction and quotas.
7	Mock interview and demo. Answer random follow-ups, practice alternatives, verify actual demo configuration.

Score yourself from 0 to 2

For each skill, 0 means cannot explain, 1 means can explain with notes, and 2 means can explain and handle a changed scenario: product value, frontend flow, backend layers, ownership, schema/migrations, token lifecycle, streaming/cancel, embeddings/retrieval, extraction/lifecycle, privacy/consent, deployment diagnosis and evaluation/scale.

A high total should come from reasoning, not memorized phrases. Any zero in ownership, authentication, consent or deployment versioning deserves another pass before the interview. Try one hostile follow-up per topic: “What if two requests race?”, “What if a secret is in a memory?”, “What if the database changes?”

Before the interview

Confirm the deployed version and provider configuration; warm free services; use a disposable account; remove secrets from screenshots; save offline diagrams; know which features are branch-only. If a demo fails, narrate the diagnostic path calmly rather than changing credentials or weakening rules live.

Final readiness check: On a blank page draw the complete browser → proxy → API → DB/memory/model path, then mark every trust boundary and every asynchronous step. If you can explain each mark, you understand the project's core.

54 • One-page recall sheet and source trail

Remember these twelve facts

- Product: compact, inspectable, user-controlled context across chats.
- Browser: React/TypeScript SPA; relative /api; access token in memory.
- Server: FastAPI routes, services, repositories and interchangeable providers.
- Auth: Argon2id; short HS256 access JWT; hashed rotating opaque refresh cookie.
- Isolation: authenticated user predicates; UUID alone is not permission.
- Storage: PostgreSQL + pgvector; SQLite for fast tests plus real PG integration.
- Chat: prepare → recall → prompt → stream → save → background extract.
- Retrieval: active owned rows; topic similarity gate then scoring/top-k.
- Broad preference fix: bounded active preference lookup for conservative explicit intent.
- Lifecycle: Auto/Ask; active/pending/superseded; versions, restore and hard deletion.
- Operations: Nginx HTTPS upstream, SNI/Host, migrations, readiness and cold-start distinction.
- Honesty: context delivery is not answer accuracy; branch is not deployment; analytics values are not full measurements.

Source trail for this edition

Primary evidence is `lingudusailokesh/memoryai` at branch commit `e945012`, read on 7 October 2026. Key anchors are in chapter 42. Supplementary project documents: README, decisions, memory-system, interview-notes, render-authentication and cross-chat-recall. Historical documents are useful context but may be stale; inspect current code before quoting them.

Relevant review links: <https://github.com/lingudusailokesh/memoryai/pull/1> (readiness handling); <https://github.com/lingudusailokesh/memoryai/pull/2> (separate account diagnosis work); <https://github.com/lingudusailokesh/memoryai/pull/3> (broad preference recall). This guide does not assert that all three are merged/deployed.

Reference documentation for continued learning

FastAPI: fastapi.tiangolo.com • React: react.dev • TypeScript: typescriptlang.org/docs • SQLAlchemy: docs.sqlalchemy.org • Alembic: alembic.sqlalchemy.org • PostgreSQL: postgresql.org/docs • pgvector: github.com/pgvector/pgvector • Nginx: nginx.org/en/docs • OWASP password storage and CSRF cheat sheets: cheatsheetseries.owasp.org.

These links are further reading, not a claim that their current pages were used as production evidence. Theory, code-inspected behavior, local verification and live verification must remain distinct.

Your strongest interview answer is an explanation you can trace to a mechanism, a test and an honest boundary. Know what the system does, why it does it, and where you would improve it next.